

A Project Report on

VRGen: Deep Learning Based VR Property Walkthroughs Using 2D Plans

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Yash Penkar: 22106118

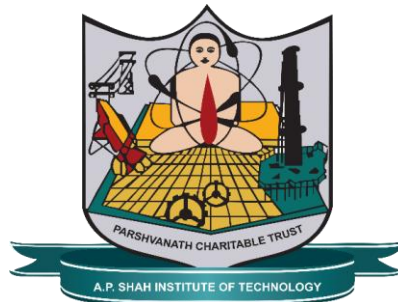
Prarthana Patil: 22106035

Rutuja Pawar: 22106043

Dhruv Sawant: 22106015

Under the Guidance of

Prof. Vijesh Mundokalam

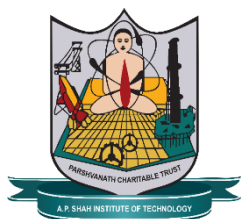


**Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)**

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026



A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled “VRGen: Deep Learning Based VR Property Walkthroughs Using 2D Plans” is a bonafide work of **Yash Penkar (22106118), Prarthana Patil (22106035), Rutuja Pawar (22106043), Dhruv Sawant (22106015)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering in Computer Science & Engineering (Artificial Intelligence & Machine Learning)**.

Prof. Vijesh Mundokalam
Guide

Prof. Sayali Badhan
Project Co-ordinator

Dr. Jaya Gupta
Head of Department

Dr. Uttam Kolekar
Principal



A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled *VRGen: Deep Learning Based VR Property Walkthroughs Using 2D Plans* by *Yash Penkar, Prarthana Patil, Rutuja Pawar, Dhruv Sawant* is approved for the degree of *Bachelor of Engineering* in *Computer Science & Engineering (Artificial Intelligence & Machine Learning)*, 2025-26.

Examiner Name

Signature

1. _____

2. _____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Yash Penkar 22106118)

(Prarthana Patil 22106035)

(Rutuja Pawar 22106043)

(Dhruv Sawant 22106015)

Date:

Abstract

In real estate visualization, interior design, and pre-selling systems, converting two-dimensional architectural floor plans into three-dimensional models is at the core of spatial understanding. However, the conventional process of modeling relies on software like CAD and Blender, which involves manual work, expertise, and time. Even slight modifications in design require extensive work and remodeling. To address the challenge of the conventional process, the present study proposes a framework that uses deep learning techniques to automatically generate three-dimensional models from two-dimensional floor plan images using conditional Generative Adversarial Networks (cGANs). Experiments were performed with architectural floor plan datasets in order to test the ability of the framework to generate consistent three-dimensional structures. The results demonstrate that the proposed method is feasible and scalable to more intricate architectural designs. This work contributes to SDG 9 (Industry, Innovation and Infrastructure) by promoting advanced technological solutions, SDG 11 (Sustainable Cities and Communities) by supporting efficient urban planning and visualization, SDG 4 (Quality Education) by enabling enhanced learning tools in architecture and design, and SDG 12 (Responsible Consumption and Production) by reducing the need for physical modeling resources. The research demonstrates the potential of deep learning techniques in facilitating scalable and sustainable architectural visualization.

Keywords: *Architectural Visualization, Floor Plan Analysis, 3D Reconstruction, Virtual Reality, Deep Learning.*

CONTENTS

1. Introduction.....	1
1.1 Need for the VRGen System	2
1.2 Overview of Conditional GAN (cGAN).....	2
1.3 System Overview	3
2. Survey	4
3. Limitation of Existing System	12
4. Problem Statement and Objective	15
4.1 Problem Statement.....	15
4.2 Objectives	16
5. Proposed System.....	18
5.1 Architecture Diagram.....	18
5.2 Data Flow Diagram.....	20
5.3 Activity Diagram	22
5.5 Use Case Diagram	26
6. Experimental Setup.....	28
6.1 Details of Database / Input Data to the System	28
6.2 Performance Evaluation Parameters (Validation Metrics)	29
6.3 Software and Hardware Setup	30
6.3.1 Software Configuration	30
6.3.2 Hardware Configuration	33
7. Results and Discussion	34
7.1 Dataset and Training Configuration.....	34
7.2 Quantitative Results (Loss Analysis).....	35
7.3 Loss Behavior Analysis	35
7.4 Qualitative Results.....	37
7.5 Comparison with Existing Systems	38
7.6 Overall Performance Evaluation.....	40
8. Conclusion and Future Work	41
8.1 Conclusion	41
8.2 Future Work	42
References.....	44
Appendix.....	46
Publication	51

LIST OF FIGURES

Figure 5.1: Overall System Architecture of VRGen.....	19
Figure 5.2: Data Flow Diagram	21
Figure 5.3: Activity Diagram.....	23
Figure 5.4: Sequence Diagram.....	25
Figure 5.5: Use Case Diagram.....	26
Figure 7.1 Pipeline for converting 2D floor plans into 3D Models.....	38

LIST OF TABLES

Table 2.1: Research Gap	9
Table 7.5.1: Comparative Advantage Over Existing Systems	39

LIST OF GRAPHS

Graph 7.1: Variation of L1 Loss and Edge Loss Across Training Epochs.....	36
---	----

ABBREVIATIONS

AI	<i>Artificial Intelligence</i>
DL	<i>Deep Learning</i>
CN	<i>Computer Network</i>
cGAN	<i>Conditional Generative Adversarial Network</i>
GAN	<i>Generative Adversarial Network</i>
GLB	<i>Binary GL Transmission Format (3D model format)</i>
RGB	<i>Red, Green, Blue (colour channels)</i>
U-Net	<i>Encoder-Decoder CNN with skip connections</i>
VR	<i>Virtual Reality</i>

Chapter 1

Introduction

1. Introduction

Architectural visualization plays a vital role in modern real estate, interior design, and construction industries by enabling stakeholders to understand spatial layouts and design concepts before actual construction. Traditionally, two-dimensional (2D) floor plans have been widely used to represent architectural designs, as they provide a simplified overview of room arrangements, walls, and structural elements. However, 2D representations lack depth perception and fail to convey a realistic sense of scale, spatial relationships, and user experience. This limitation makes it difficult for non-technical stakeholders, such as clients and property buyers, to accurately interpret designs, often leading to miscommunication and inefficient decision-making.

To overcome these limitations, three-dimensional (3D) visualization techniques have gained importance, as they provide a more immersive and realistic representation of architectural spaces. Despite their advantages, conventional 3D modeling approaches rely heavily on manual tools such as CAD and Blender. These methods are time-consuming, require skilled professionals, and are not suitable for rapid design iterations. Additionally, even minor changes in floor plans demand significant rework, making the process inefficient and costly for large-scale or real-time applications.

Recent advancements in deep learning and computer vision have opened new possibilities for automating the transformation of 2D floor plans into 3D representations. In particular, image-to-image translation techniques enable models to learn spatial and structural features directly from floor plan images. At the same time, virtual reality (VR) technologies provide an interactive and immersive platform for visualizing architectural environments, improving user engagement and understanding.

This project, VRGen, proposes an automated end-to-end system that converts 2D floor plans into interactive 3D VR walkthroughs using deep learning techniques. The system leverages Conditional Generative Adversarial Networks (cGANs) to generate heightmaps from floor plans, which are further processed and transformed into 3D meshes suitable for VR environments. By integrating deep learning with immersive visualization, the proposed system reduces manual effort, enhances scalability, and improves the overall efficiency of architectural visualization workflows.

1.1 Need for the VRGen System

The need for the VRGen system arises from the limitations of traditional architectural workflows. Manual 3D modeling is time-intensive, expensive, and dependent on expert knowledge, making it unsuitable for rapid prototyping and real estate pre-selling scenarios. Additionally, 2D floor plans are often difficult for non-technical users to interpret, leading to misunderstandings and delayed decision-making. There is a growing demand for systems that can automatically generate realistic and interactive 3D visualizations from simple inputs. The VRGen system addresses these challenges by enabling quick generation of 3D walkthroughs, improving communication between stakeholders, reducing dependency on manual processes, and supporting scalable deployment in real-world applications.

1.2 Overview of Conditional GAN (cGAN)

The proposed system is based on deep learning, specifically a Conditional Generative Adversarial Network (cGAN), which is widely used for image-to-image translation tasks. A cGAN consists of two main components: a generator and a discriminator, trained in an adversarial manner.

The generator learns to transform input floor plan images into corresponding heightmaps that encode spatial and structural information, while the discriminator evaluates the realism of the

generated outputs. Unlike traditional GANs, cGANs are conditioned on the input data, ensuring that the generated output is directly aligned with the given floor plan.

In this system, the generator is implemented using an enhanced U-Net architecture with attention mechanisms to capture important architectural features such as walls and room boundaries. A PatchGAN discriminator is used to ensure local structural consistency. Multiple loss functions, including L1 loss, edge loss, perceptual loss, and total variation loss, are incorporated to improve reconstruction accuracy, edge sharpness, and smoothness of the generated heightmaps.

1.3 System Overview

The VRGen system follows an end-to-end automated pipeline for converting 2D floor plans into immersive 3D environments. The process begins with preprocessing of input images, including resizing, normalization, contrast enhancement, and edge detection to highlight architectural features. The processed images are then passed through the trained deep learning model, which generates heightmaps representing spatial depth information. These heightmaps undergo post-processing steps such as noise removal, wall enhancement, floor leveling, and morphological operations to improve structural quality.

Finally, the refined heightmaps are converted into 3D meshes using geometric reconstruction techniques and exported in formats suitable for VR visualization. The resulting models can be explored interactively, providing users with a realistic walkthrough experience. This automated pipeline minimizes human intervention while ensuring efficiency, scalability, and accurate spatial representation, making it suitable for applications in real estate, architecture, and interior design.

Chapter 2

Literature Survey

2. Survey

1. Learning to Predict 3D Meshes from a Single Image via Depth Consistency: Chinese Journal of Mechanical Engineering, 2025

This paper presents a novel deep learning method for reconstructing 3D meshes from a single 2D image. The approach focuses on maintaining depth consistency to improve the accuracy of reconstructed shapes. It uses neural networks to learn strong shape priors without requiring explicit 3D annotations. The method employs differentiable rendering techniques to generate depth maps from predicted meshes. Additional loss functions are introduced to refine mesh structure and preserve fine details. Compared to traditional voxel or point cloud methods, mesh representation provides better surface quality. The model performs well on both synthetic and real-world datasets. This approach is highly useful in applications like 3D modeling, AR/VR, and computer graphics [1].

2. GenAICHI 2025: Generative AI and HCI at CHI 2025: Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI EA '25), 2025

This paper serves as an overview for a specialized workshop focused on the intersection of Generative Artificial Intelligence and Human-Computer Interaction (HCI). It highlights the

rapid evolution of generative models and their profound impact on how users interact with technology and create content. The workshop aims to bring together researchers to discuss the design, evaluation, and ethical implications of generative systems in various domains. Key themes include user agency, the collaborative potential between humans and AI, and the necessity for transparency in automated creative processes. The publication documents the collective effort to establish a research agenda that prioritizes human-centric approaches in the development of AI tools. It also addresses the challenges of maintaining accountability and preventing misinformation in a generative-heavy digital landscape [2].

3. Plan–MORPH: Conversion of 2D Floorplans to 3D Models: International Journal of Enhanced Research in Science, Technology & Engineering, 2024

This paper presents an automated system that converts 2D floorplan images into 3D models using image processing and modeling techniques. The system is implemented using Python with Flask as the backend framework. It utilizes OpenCV for image preprocessing tasks such as edge detection, contour extraction, and noise reduction. The extracted structural features like walls and boundaries are then processed using the Trimesh library to generate 3D geometry. The output is exported in STL format, which can be viewed in standard 3D viewers. The system improves efficiency in architectural design by reducing manual effort and enhancing visualization. It also provides a user-friendly interface for uploading images and downloading models. The proposed method supports better spatial understanding and decision-making in design workflows [3].

4. Automatic Reconstruction of Semantic 3D Models from 2D Floor Plans: Computer Vision, 2023

This paper presents an automated approach for converting 2D floor plans into semantic 3D building models. The method uses deep learning techniques, particularly CNN-based segmentation, to identify structural elements such as walls, doors, and windows. After segmentation, the extracted features are converted into vectorized representations. These representations are then used to generate accurate 3D models of buildings. The system reduces the need for manual drafting and speeds up the digitalization process. It performs well on datasets like CUBIcasa5k and generalizes across different plan styles. The pipeline also supports exporting models into formats used in CAD and BIM systems. Overall, it significantly improves efficiency in architectural modeling workflows [4].

5. The Implementation of Virtual Reality (VR) Technology in Real Estate Industry: Planning Malaysia Journal, 2023

This paper examines the growing role of Virtual Reality (VR) in the real estate industry, especially after the COVID-19 pandemic. It highlights how VR enables virtual property tours, allowing buyers to explore properties remotely. The study identifies key VR features such as 360-degree visualization and interactive room layouts. These features enhance customer engagement and provide a realistic viewing experience. The research is based on data collected from real estate professionals and analyzed using statistical tools. It shows that VR reduces the need for physical visits and improves marketing efficiency. Additionally, VR helps buyers make better decisions by offering detailed property insights. The study concludes that VR is a valuable tool for modern real estate marketing [5].

6. Generating 3D Architected Materials Using Diffusion Models Based on Language Cues, 2022

This paper explores the use of diffusion models (such as Stable Diffusion) for generating 3D material structures from textual descriptions. It converts generated 2D images into voxel-based 3D representations and further into mesh models for simulation or manufacturing. The approach demonstrates how modern generative models can bridge text, image, and 3D design domains. Although not directly focused on architecture, it introduces innovative methods for transforming 2D representations into 3D structures using deep learning [6].

7. 3DPlanNet: Generating 3D Models from 2D Floor Plan Images Using Ensemble Methods, 2021

This paper proposes a hybrid approach combining deep learning and rule-based heuristic methods to convert 2D floor plan images into 3D models. The system, called 3DPlanNet, uses a small dataset (only 30 images) and still achieves high accuracy by integrating pattern recognition, object detection, and heuristic corrections. It generates structured 3D vector data by identifying walls, doors, windows, and rooms, achieving over 95% wall accuracy. The method addresses the limitation of large data requirements in deep learning and demonstrates scalability by converting over 110,000 floor plans into 3D models [7].

8. Automatic 3D Building Model Generation Using Deep Learning Methods Based on CityJSON and 2D Floor Plans, 2021

This paper presents a deep learning-based framework for generating 3D building models by integrating 2D floor plans with external 3D city data (CityJSON). It extracts semantic

information such as walls, openings, and room labels from floor plans using techniques like semantic segmentation and object detection. The system then performs data fusion to combine interior and exterior information into complete 3D models. The approach is significant for urban planning and building lifecycle management, though it requires complex preprocessing and multiple data sources [8].

9. Developing and Evaluating a Virtual Reality-Based Navigation System for Pre-Sale Housing Sales, 2018

This study focuses on the application of virtual reality (VR) in the real estate sector, particularly for pre-sale housing. It develops a VR-based navigation system that allows users to experience unconstructed buildings in an immersive environment. The results show that VR improves users' understanding of spatial layouts and increases their intention to purchase properties. However, usability challenges and user familiarity with VR technology remain limitations. The paper highlights the importance of VR in enhancing visualization and decision-making in architectural and construction domains [9].

10. Image-to-Image Translation with Conditional Adversarial Networks, 2017

This paper introduces conditional Generative Adversarial Networks (cGANs) as a general framework for image-to-image translation tasks. The model learns a mapping between input and output images while simultaneously learning a loss function, enabling generation of realistic outputs without handcrafted objectives. It uses a U-Net-based generator and a PatchGAN discriminator to preserve structural details and improve image quality. The approach is widely applicable to tasks such as converting labels to images, edges to photos, and maps to aerial views, making it foundational for applications like floor plan to 3D/heightmap generation [10].

11. Perceptual Losses for Real-Time Style Transfer and Super-Resolution: arXiv preprint, 2016

This paper proposes a method for image transformation tasks by combining the benefits of feed-forward convolutional neural networks and perceptual loss functions. Instead of relying on per-pixel loss, the authors use high-level features extracted from pretrained networks to define a loss function that captures semantic and perceptual details. This approach is applied to real-time style transfer and single-image super-resolution with significant performance gains. For style transfer, the feed-forward network achieves qualitative results similar to existing optimization-based methods but operates three orders of magnitude faster. In super-resolution,

the use of perceptual loss results in images that are more visually pleasing and detailed compared to those trained with pixel-level metrics. The work emphasizes the trade-off between speed and visual quality in deep learning-based image synthesis [11].

12. U-Net: Convolutional Networks for Biomedical Image Segmentation: Computer Vision and Pattern Recognition, 2015

This paper proposes the U-Net architecture, a powerful convolutional neural network designed specifically for image segmentation tasks. It follows an encoder-decoder structure where the contracting path captures context and the expanding path enables precise localization. A key feature of U-Net is the use of skip connections, which combine low-level and high-level features for better accuracy. The model performs well even with limited training data due to strong data augmentation techniques. It achieves high performance in pixel-level classification tasks. U-Net is widely used in medical imaging and computer vision applications. Its ability to extract fine structural details makes it suitable for floor plan analysis. This is particularly useful in detecting walls, rooms, and architectural components [12].

13. Integrated Virtual Reality and Discrete Event Simulation Methods for Production System Research in Construction: Construction Applications of Virtual Reality Conference, 2013

This paper explores the integration of Virtual Reality (VR) and Discrete Event Simulation (DES) for construction system analysis. It introduces a hybrid virtual environment where users can interact with simulated construction processes. The system allows researchers to study decision-making behavior in a controlled yet realistic setting. Unlike traditional simulations, this approach incorporates human interaction, making results more practical and reliable. It helps evaluate construction workflows, resource allocation, and production efficiency. The virtual environment enables real-time experimentation without affecting actual construction sites. This reduces risks and costs associated with real-world testing. The study demonstrates the effectiveness of VR in improving construction planning and management [13].

14. Nonlinear Total Variation Based Noise Removal Algorithms: North-Holland (Physical Journal), 1992

This paper proposes a nonlinear optimization-based approach for removing noise from images using total variation (TV) minimization. The method formulates image denoising as a constrained optimization problem where noise statistics are incorporated using Lagrange multipliers. Unlike traditional least squares (L2 norm) techniques, the authors emphasize the

effectiveness of the L1 norm for preserving edges while removing noise. The solution is obtained through a gradient projection method, leading to a time-dependent partial differential equation. The algorithm iteratively converges to a steady-state denoised image. It is computationally efficient and produces high-quality results even for heavily noisy images. The approach is particularly useful in maintaining sharp discontinuities and reducing oscillatory artifacts. This work laid the foundation for modern PDE-based image denoising techniques [14].

15. An Isotropic 3x3 Image Gradient Operator (Sobel Operator): Presented at Stanford Artificial Intelligence Project, 1968 (Referenced in Pattern Classification and Scene Analysis, 1975)

This work introduces the Sobel operator, a widely used edge detection technique in image processing. The operator estimates the gradient of image intensity using a 3x3 neighborhood, providing both magnitude and direction of edges. It improves upon earlier methods like the Roberts Cross operator by offering better isotropy and noise resistance. The method computes gradients using weighted differences in pixel intensities along horizontal and vertical directions. It is computationally efficient and suitable for real-time applications. The Sobel operator is widely used in computer vision tasks such as feature extraction, object detection, and boundary detection. It plays a fundamental role in preprocessing steps of many image analysis systems. This operator remains a cornerstone technique in digital image processing [15].

Table 2.1: Research Gap

Research Paper	Year	Algorithm	Gap
Learning to Predict 3D Meshes via Depth Consistency	2025	Deep Learning + Differentiable Rendering	Requires high computational power; needs large training data
GenAICHI 2025: Generative AI and HCI	2025	Generative AI + HCI Frameworks	Conceptual work; lacks implementation details; no direct application to 3D modeling
Plan-MORPH: 2D Floorplans to 3D Models	2024	Image Processing (OpenCV) + Trimesh Modeling	Lacks deep learning; limited adaptability to complex designs

Automatic Reconstruction of Semantic 3D Models from Floor Plans	2023	CNN-based Segmentation + Vectorization	Errors in complex layouts; depends heavily on dataset quality
VR Technology in Real Estate Industry	2023	VR Systems + Statistical Analysis	No technical model for automation; limited to visualization and marketing
3D Architected Materials using Diffusion Models	2022	Diffusion Models (Text-to-Image + Voxel/3D Conversion)	Not architecture-specific; computationally intensive; indirect 3D generation
3DPlanNet: 2D to 3D using Ensemble Methods	2021	Hybrid (Deep Learning + Heuristic Rules + Pattern Recognition)	Small dataset; rule-based components limit generalization
3D Building Model Generation using CityJSON + DL	2021	Deep Learning (Semantic Segmentation + Data Fusion)	Requires multiple data sources; complex preprocessing pipeline
VR-Based Navigation System for Pre-Sale Housing	2018	Virtual Reality Visualization System	Limited scalability; usability issues; depends on hardware and user familiarity
Image-to-Image Translation with cGANs	2017	Conditional GAN (U-Net Generator + PatchGAN Discriminator)	Training instability; requires large datasets; may produce blurry outputs
Perceptual Losses for Style Transfer and Super-Resolution	2016	CNN + Perceptual Loss (VGG-based feature loss)	Focuses on visual quality, not structural accuracy; not directly applicable to 3D reconstruction

U-Net: Convolutional Networks for Biomedical Image Segmentation	2015	CNN (Encoder-Decoder with Skip Connections)	Requires labeled data; struggles with highly complex or noisy floor plans
Integrated VR and Discrete Event Simulation for Construction	2013	VR + Discrete Event Simulation (DES)	High implementation complexity; lacks automation; requires manual modeling
Nonlinear Total Variation Based Noise Removal Algorithms	1992	Total Variation (TV) Minimization, PDE-based optimization	Computationally expensive; not suitable for real-time applications; may over smooth fine textures
A 3×3 Isotropic Gradient Operator for Image Processing (Sobel Operator)	1968	Sobel Edge Detection Operator (gradient-based filtering)	Sensitive to noise; limited accuracy for complex structures; not suitable for high-level feature extraction

Chapter 3

Limitation of Existing System

3. Limitation of Existing System

- **Limited Low-Level Feature Capability:** Early methods like the Sobel operator rely only on gradient-based edge detection, which captures basic intensity changes but fails to understand higher-level structures. These approaches cannot differentiate between meaningful architectural elements and noise. They are highly sensitive to variations in lighting and image quality. As a result, they are not suitable for complex floor plan interpretation. This limits their application in modern computer vision tasks requiring semantic understanding.
- **Over-Smoothing and Computational Cost:** Total Variation (TV) based denoising methods tend to remove noise effectively but often oversmooth important fine details. This results in the loss of critical structural information required for tasks like floor plan analysis. Additionally, these methods rely on iterative optimization, making them computationally expensive. They are not suitable for real-time processing or large-scale applications. This limits their usability in modern deep learning pipelines.

- **Lack of Real-Time and Automated Integration:** VR and Discrete Event Simulation-based systems require significant manual setup and modeling. They lack automation in generating environments, making them time-consuming and resource-intensive. Real-time interaction is limited due to computational constraints. These systems are often designed for analysis rather than automated generation. This reduces their applicability in scalable architectural modeling workflows.
- **Dependency on Labeled Data:** Deep learning models like U-Net require large amounts of annotated data for effective training. Creating such labeled datasets for floor plans is time-consuming and expensive. The model may also struggle when exposed to unseen or complex layouts. Performance degrades in the presence of noise or irregular designs. This limits generalization across diverse architectural styles.
- **Lack of Structural Understanding:** Perceptual loss-based models focus primarily on visual quality rather than structural correctness. While outputs may look realistic, they may not preserve geometric or spatial relationships. This makes them unsuitable for tasks like 3D reconstruction from floor plans. The approach does not explicitly enforce architectural constraints. Hence, it lacks reliability for engineering applications.
- **Training Instability and Data Dependency:** Conditional GANs (cGANs) suffer from unstable training and issues like mode collapse. They require large and diverse datasets to produce reliable outputs. The generated images may sometimes lack consistency or contain artifacts. Fine-tuning GANs is also challenging and resource-intensive. These limitations reduce their robustness in real-world applications.
- **Usability and Hardware Constraints in VR:** VR-based navigation systems depend heavily on user familiarity and specialized hardware. Many users may find VR interfaces difficult to use initially. The cost of VR equipment can also be a barrier. Additionally, these systems focus on visualization rather than automated model generation. This limits their scalability in practical deployments.
- **Limited Generalization in Hybrid Models:** Hybrid approaches like 3DPlanNet combine deep learning with rule-based heuristics. While effective on small datasets, rule-based components restrict adaptability. These systems may fail when encountering unseen layouts or styles. The reliance on handcrafted rules reduces flexibility. This limits scalability across diverse real-world data.
- **High Complexity and Data Dependency:** Frameworks that integrate multiple data sources like CityJSON require complex preprocessing. They depend on the availability of external datasets, which may not always be accessible. Data fusion increases system

complexity and error propagation. Maintaining consistency between 2D and 3D data is challenging. This makes the system difficult to deploy in practical scenarios.

- **Indirect 3D Generation and High Computation:** Diffusion-based models generate 3D structures indirectly through intermediate 2D representations. This multi-step process increases computational cost and time. The models require significant hardware resources for training and inference. They are not specifically designed for architectural applications. This limits their direct usability in floor plan-based modeling.
- **Errors in Complex Layout Interpretation:** CNN-based reconstruction methods may struggle with complex or cluttered floor plans. Misclassification of walls, doors, and rooms can lead to inaccurate 3D models. The performance heavily depends on the quality of input data. Generalization across different styles remains a challenge. This affects the reliability of automated systems.
- **Lack of Technical Depth in VR Applications:** Studies on VR in real estate mainly focus on user experience and marketing benefits. They do not provide technical solutions for automated model generation. The research lacks integration with AI-based reconstruction methods. This creates a gap between visualization and actual model creation. Hence, their contribution is limited to presentation rather than automation.
- **Limited Adaptability of Traditional Image Processing:** Systems like Plan-MORPH rely on traditional image processing techniques such as edge detection and contour extraction. These methods lack the ability to learn from data. They perform poorly on complex or noisy floor plans. Adaptability to different layouts is limited. This reduces their effectiveness compared to deep learning approaches.
- **High Computational Requirements in 3D Reconstruction:** Advanced deep learning models for 3D mesh generation require significant computational resources. Training such models demands large datasets and powerful GPUs. The inference process can also be slow. This limits real-time applications and accessibility. It becomes challenging to deploy these models in resource-constrained environments.
- **Lack of Practical Implementation in Conceptual Frameworks:** Research like GenAICHI focuses on conceptual discussions around Generative AI and HCI. While it provides valuable insights, it lacks concrete implementation details. There is no direct application to specific domains like floor plan reconstruction. The absence of experimental validation limits its practical usefulness. This creates a gap between theory and real-world deployment.

Chapter 4

Problem Statement and Objective

4. Problem Statement and Objective

4.1 Problem Statement

Real estate developers often face significant challenges in marketing properties during the early stages of construction. This is especially true before sample flats or physical models are available. Traditional tools like brochures, static images, and 2D floor plans offer limited visualization. They fail to represent the true spatial layout, depth, and scale of a property. As a result, buyers struggle to accurately imagine the final outcome. This reduces their confidence in making early investment decisions. Additionally, 2D plans do not clearly show room connectivity, dimensions, lighting, or aesthetics. Buyers are forced to rely on imagination, leading to misunderstandings and mismatched expectations. This gap between visualization and reality affects decision-making and marketing effectiveness. Hence, a solution using 3D models and Virtual Reality is needed to improve visualization, engagement, and early-stage sales.

4.2 Objectives

- **To build a floorplan-to-mesh pipeline:** Develop an end-to-end system that converts 2D floor plan images into structured 3D mesh models. The pipeline will process input images, extract architectural features, and generate corresponding 3D representations. It ensures a smooth transformation from 2D layouts to spatial 3D structures. This automation reduces manual effort and improves efficiency in model generation.
- **To implement automated feature extraction:** Design and integrate deep learning models to automatically detect key elements such as walls, doors, windows, and rooms. This eliminates the need for manual annotation and rule-based detection methods. The system will improve accuracy in identifying structural components. It also ensures adaptability across different floor plan styles and layouts.
- **To generate realistic 3D visualizations:** Create 3D models that accurately represent spatial layout, proportions, and depth of the floor plan. Techniques like heightmap prediction and mesh generation will enhance realism. The goal is to provide users with a clear and detailed understanding of the property. This improves visualization compared to traditional 2D representations.
- **To integrate Virtual Reality (VR) for immersive experience:** Enable users to explore the generated 3D models in a virtual environment. The VR integration allows interactive navigation through rooms and spaces. This enhances user engagement and spatial understanding. It provides a real-world experience before actual construction is completed.
- **To improve generalization across diverse floor plans:** Develop a system capable of handling various architectural styles, layouts, and complexities. The model should perform consistently on unseen data and different formats of floor plans. This ensures robustness and real-world applicability. It also reduces dependency on specific datasets.
- **To optimize computational efficiency and resource usage:** Design the system to work efficiently with minimal computational resources and processing time. Optimization techniques will reduce model complexity without compromising accuracy. This makes the system practical for real-time and large-scale applications. It also ensures usability in resource-constrained environments.
- **To enhance customer engagement and decision-making:** Provide an interactive platform where users can visualize and explore properties before construction. This improves buyer confidence and reduces uncertainty in decision-making. The system bridges the gap between imagination and reality. Ultimately, it helps boost early-stage property sales and marketing effectiveness.

- **To allow future extensibility and enhancements:** Design the system architecture in a modular way so that new features like furniture placement, lighting simulation, and texture enhancement can be added. This ensures long-term usability of the system. It also supports research extensions and improvements. The system remains scalable and adaptable.
- **To incorporate height estimation using deep learning (cGAN):** Integrate a conditional Generative Adversarial Network (cGAN) to predict heightmaps from 2D floor plans. This adds depth information to the model and improves the realism of 3D reconstruction. The generator learns spatial depth while the discriminator ensures quality. This enhances visual accuracy compared to basic extrusion methods.
- **To enable real-time or near real-time processing:** Design the system to process floor plans quickly and generate 3D outputs with minimal delay. This is important for practical applications where users expect fast results. Optimization techniques will be applied to reduce latency. This improves usability in real-world scenarios.

Chapter 5

Proposed System

5. Proposed System

The proposed VRGen system is a deep learning–based framework designed to convert 2D floorplan images into immersive 3D virtual reality walkthroughs. The system utilizes an Enhanced U-Net Generator within a Generative Adversarial Network (GAN) architecture to generate heightmaps representing structural elements such as walls and floors. These heightmaps are further refined using post-processing techniques and finally converted into 3D meshes, which are exported for VR visualization.

5.1 Architecture Diagram

The architecture of the VRGen system is divided into multiple interconnected stages, beginning with input acquisition and ending with the final VR-based visualization output. Each stage is designed to handle a specific part of the pipeline, ensuring a smooth and efficient transformation of data. The system integrates preprocessing techniques to prepare and enhance the input images, deep learning models to extract structural information and generate heightmaps, and 3D reconstruction modules to convert the processed data into realistic 3D models.

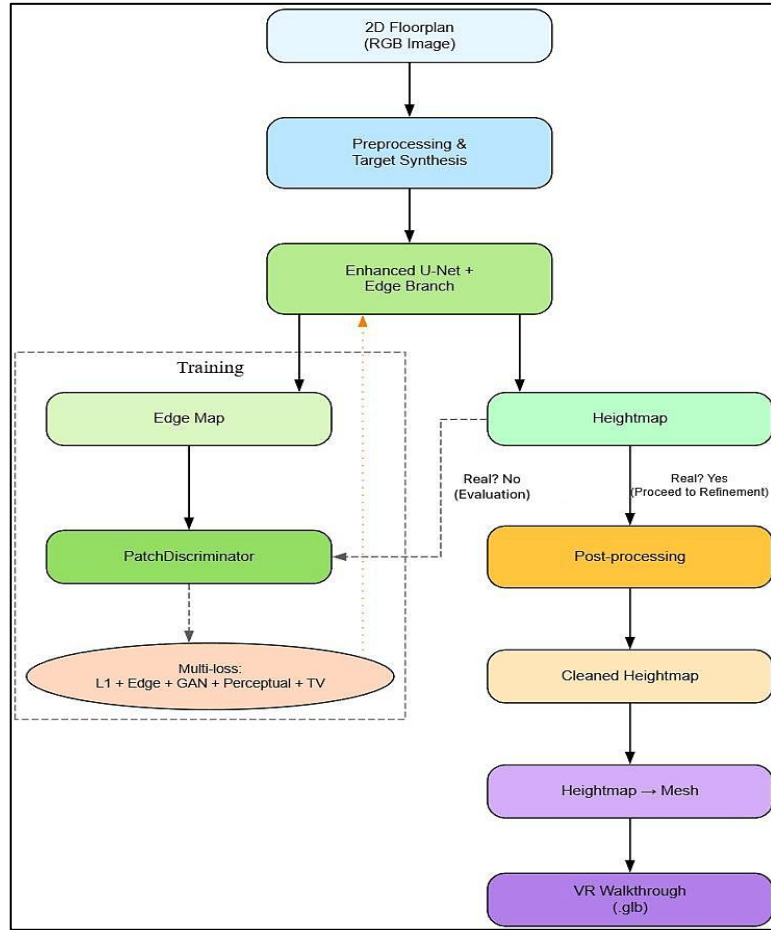


Figure 5.1: Overall System Architecture of VRGen

Main Components:

- **Input Module:** The input module is responsible for receiving floorplan images from the user. It supports commonly used image formats such as PNG and JPG, ensuring compatibility and ease of use. This flexibility allows users to provide architectural plans from various sources without requiring format conversion.
- **Preprocessing Module:** The preprocessing module prepares the input data for model processing by performing several essential operations. It resizes all images to a fixed dimension of 256×256 pixels for consistency, applies normalization and data augmentation techniques to improve model generalization, and enhances edges to ensure clearer detection of walls and structural elements.
- **Dataset Loader:** The dataset loader handles the organization and loading of data into the system. It loads floorplan images along with their corresponding target heightmaps when available. In cases where target data is missing, it automatically synthesizes heightmaps from the input images, ensuring uninterrupted training and inference workflows.

- **Deep Learning Model (GAN Architecture):** The system uses a Generative Adversarial Network (GAN) architecture consisting of two key components. The Generator (Enhanced U-Net) utilizes an encoder–decoder structure with attention mechanisms to effectively capture architectural features and produces both heightmap and edge predictions. The Discriminator (PatchGAN) evaluates the generated outputs by distinguishing between real and fake samples, thereby improving the realism and accuracy of the generated heightmaps.
- **Post-processing Module:** The post-processing module refines the generated heightmaps to improve structural quality. It removes noise using advanced filtering techniques, enhances important architectural features such as walls and doors, and applies morphological operations to ensure cleaner, sharper, and more consistent structural representations.
- **Heightmap Enhancement:** The heightmap enhancement stage ensures clear differentiation between structural elements. It separates walls and floors using thresholding techniques, assigning higher values to walls and lower values to floors, which is essential for accurate 3D reconstruction.
- **Mesh Generation:** The mesh generation module converts the processed heightmap into a 3D structure. It uses the marching cubes algorithm to generate a 3D mesh representation, enabling realistic visualization of the architectural layout.
- **Output Module:** The output module handles the final stage of the system by exporting the generated 3D model in .glb format. This format supports efficient rendering and enables immersive VR walkthrough visualization for end users.

The architecture ensures a smooth pipeline from image input to realistic 3D output by combining deep learning with geometric processing.

5.2 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how data moves through the different modules of the VRGen system, showing the path taken by the input as it is processed and transformed at each stage. It focuses on the transformation of 2D floorplan images into 3D models by highlighting how raw input data is converted into meaningful intermediate outputs such as preprocessed images, heightmaps, and refined structures. The DFD also helps in understanding how each module interacts with others, ensuring smooth data transfer and processing throughout the system. By representing inputs, processes, and outputs in a structured manner, it provides a clear view of the overall data pipeline and the flow of information within the system. These components work together in a coordinated manner to improve accuracy, preserve architectural

details, and produce visually convincing results suitable for immersive virtual reality walkthroughs.

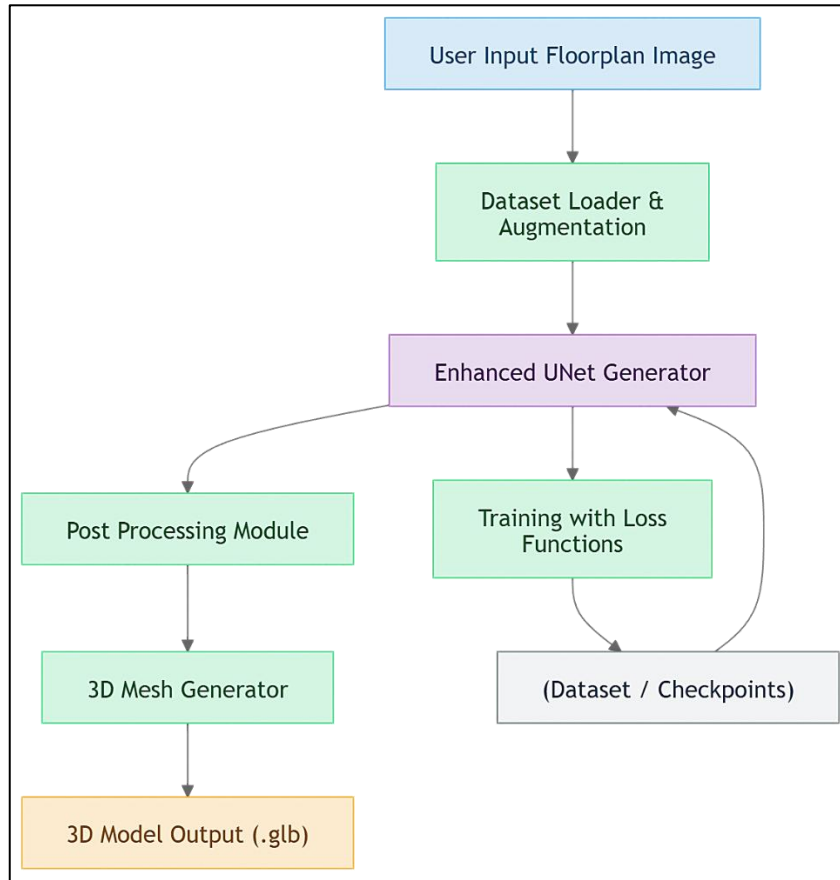


Figure 5.2: Data Flow Diagram

Data Flow Steps:

- **User uploads floorplan image:** The process begins when the user provides a 2D architectural floorplan image as input to the system. The image can be in standard formats such as PNG or JPG and represents the layout of a building, including walls, rooms, and other structural elements.
- **Image is processed in preprocessing module:** The uploaded image is passed to the preprocessing module, where it is resized to a uniform resolution (256×256 pixels) to match model requirements. Normalization is applied to standardize pixel values, and augmentation techniques such as flipping and rotation may be used to improve robustness. Additionally, edge enhancement is performed to highlight important structural features like walls.

- **Processed data is fed into the trained GAN model:** The preprocessed image is then provided as input to the trained GAN-based deep learning model. This model has already learned the mapping between floorplan images and their corresponding heightmaps during training, allowing it to generate meaningful structural representations.
- **Model generates a heightmap:** The Generator (Enhanced U-Net) processes the input image and produces a heightmap, which is a grayscale representation where pixel intensity corresponds to height values. Higher values typically represent walls, while lower values represent floors. The model may also generate edge predictions to better capture structural boundaries.
- **Heightmap is refined using post-processing:** The generated heightmap undergoes post-processing to improve quality and accuracy. This includes noise removal using filtering techniques, enhancement of architectural features such as walls and doors, and application of morphological operations to ensure smooth and continuous structures.
- **Enhanced heightmap is converted into a 3D mesh:** The refined heightmap is then transformed into a 3D model using the marching cubes algorithm. This process converts the 2D heightmap into a volumetric representation, generating vertices and surfaces that form a realistic 3D mesh of the building layout.
- **Final output is sent to VR viewer:** The generated 3D mesh is exported in .glb format and delivered to the VR viewer. This allows users to explore the building through an interactive virtual walkthrough, providing an immersive visualization of the architectural design.

The DFD highlights the flow of data rather than control logic, showing how each module transforms the input into the final output.

5.3 Activity Diagram

The activity diagram represents the step-by-step workflow of the VRGen system, including decision-making during processing.

Workflow Steps:

- **Start the process:** The workflow begins when the system is initiated by the user or application. All required modules and configurations are loaded to prepare for processing the input data.
- **Upload floorplan image:** The user provides a 2D floorplan image as input to the system. This image contains the layout of the building, including walls, rooms, and other architectural features, and is typically in formats such as PNG or JPG.

- **Perform preprocessing:** The uploaded image undergoes preprocessing, where it is resized to a fixed resolution (256×256 pixels) for consistency. Normalization is applied to standardize pixel values, and augmentation techniques may be used. Edge enhancement is also performed to highlight important structural elements like walls.

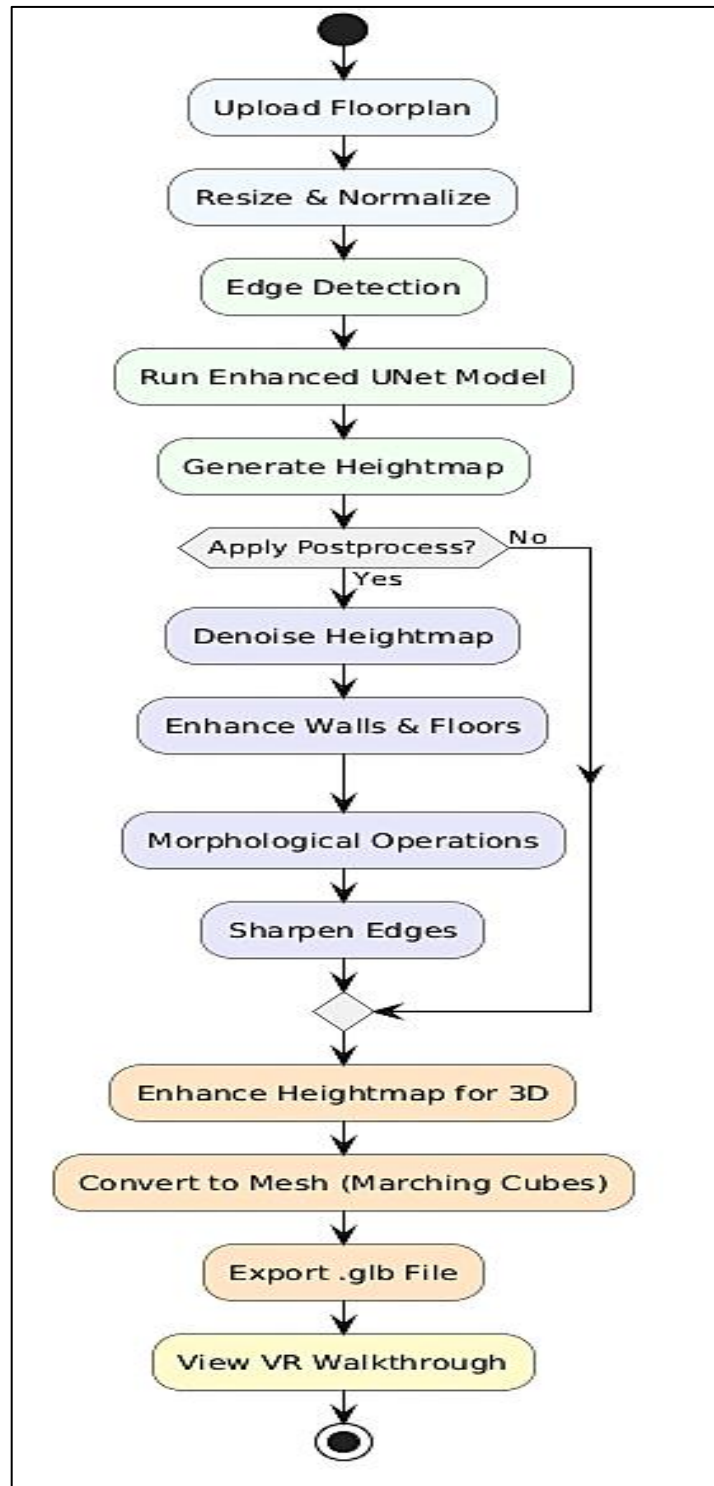


Figure 5.3: Activity Diagram

- **Run Enhanced U-Net model:** The preprocessed image is passed through the Enhanced U-Net Generator model. This deep learning model uses an encoder–decoder architecture with attention mechanisms to extract features and generate structural representations of the floorplan.
- **Generate heightmap and edge prediction:** The model outputs a heightmap, which represents the elevation of different regions (walls vs floors), and an edge prediction map, which highlights structural boundaries such as walls, doors, and windows for improved accuracy.
- **Decision: Apply Post Processing:** At this stage, a decision is made whether to apply post-processing to the generated heightmap. This decision depends on the desired output quality and processing requirements.
 - **Yes → Clean heightmap:** If post-processing is applied, the heightmap is refined using techniques such as noise removal, feature enhancement, morphological operations, and edge sharpening to produce a cleaner and more accurate structural representation.
 - **No → Skip cleaning:** If post-processing is not applied, the raw heightmap generated by the model is used directly for further processing without refinement.
- **Enhance heightmap (floor/wall separation):** The heightmap is further enhanced by applying thresholding techniques to clearly distinguish between floors and walls. Floors are assigned lower height values, while walls are assigned higher values to ensure proper 3D representation.
- **Generate 3D mesh:** The enhanced heightmap is converted into a 3D mesh using the marching cubes algorithm. This step creates a geometric representation of the building structure with vertices and surfaces.
- **Export model as .glb:** The generated 3D mesh is exported in .glb format, which is suitable for efficient rendering and supports integration with VR systems.
- **End process:** The workflow concludes after successfully generating and exporting the 3D model, which is now ready for visualization in a VR environment.

This diagram clearly shows the control flow and includes decision branching, making it useful for understanding system execution.

4. Sequence Diagram

The sequence diagram shows the interaction between different system components over time, illustrating how messages and data are exchanged from the moment the user provides input to the generation of the final output. It highlights the order in which operations are performed, including how the input is passed through preprocessing, deep learning, post-processing, and 3D generation modules. By representing the flow of communication between these components step by step, the diagram helps in understanding the dynamic behavior of the system and how each module collaborates to complete the overall process efficiently.

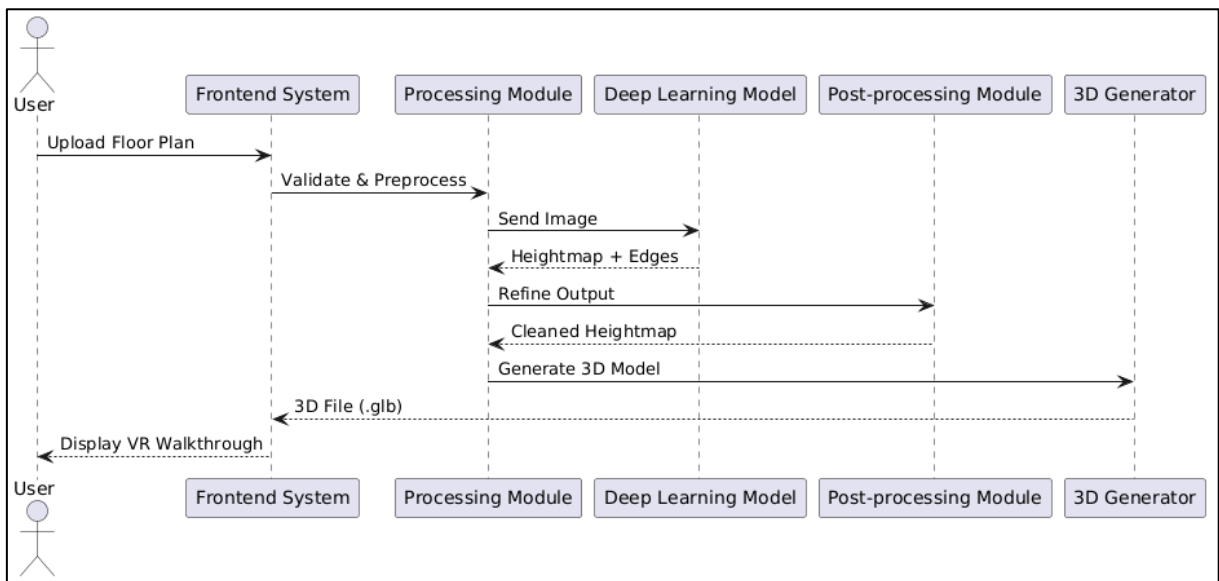


Figure 5.4: Sequence Diagram

Interaction Flow:

- **User uploads floorplan image:** The interaction begins when the user provides a 2D floorplan image to the system. This input represents the architectural layout and is uploaded through the user interface in supported formats such as PNG or JPG.
- **System sends image to preprocessing module:** Once the image is received, the system forwards it to the preprocessing module. This module prepares the image by resizing it to a standard resolution, normalizing pixel values, and enhancing important features such as edges to improve structural clarity.
- **Preprocessed data is passed to the model:** After preprocessing, the refined image data is passed to the trained deep learning model. This step ensures that the input is in the correct format and quality required for accurate prediction.

- **Model generates heightmap:** The Enhanced U-Net Generator processes the input and produces a heightmap, which represents the structural layout in terms of height values. This output highlights walls, floors, and other architectural elements. The model may also generate an edge map to improve feature detection.
- **Post-processing module refines output:** The generated heightmap is then sent to the post-processing module, where it is refined to improve quality. Noise is reduced, edges are sharpened, and architectural features such as walls and doors are enhanced using filtering and morphological operations.
- **Mesh generation module converts it to 3D:** The refined heightmap is forwarded to the mesh generation module. Here, algorithms like marching cubes are used to convert the 2D heightmap into a 3D mesh structure, creating a realistic geometric representation of the building.
- **Final output is returned to user:** The generated 3D model is exported in .glb format and delivered back to the user. The user can then view and interact with the model in a VR environment, enabling an immersive walkthrough experience.

This diagram emphasizes communication between components and the order in which operations occur.

5.5 Use Case Diagram

The use case diagram illustrates the interaction between the user and the VRGen system by highlighting the key functionalities provided by the system. It focuses on what actions the user can perform and how the system responds to those actions. This diagram helps in understanding the system from an end-user perspective rather than internal processing.

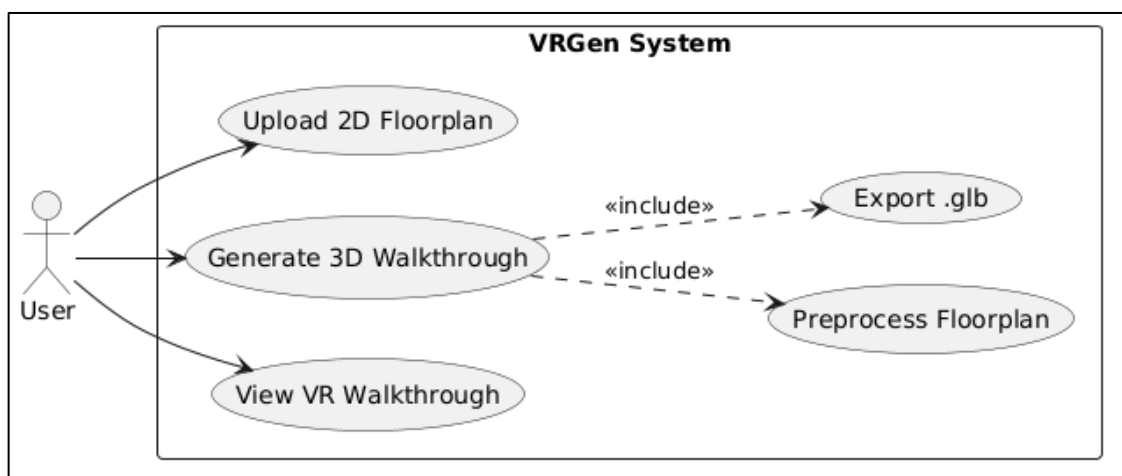


Figure 5.5: Use Case Diagram

Main Use Cases:

- **Upload Floorplan Image:** The user initiates the process by uploading a 2D floorplan image into the system. This serves as the primary input required for generating the 3D model.
- **Generate 3D Walkthrough:** After uploading the image, the user triggers the system to process the input. The system performs preprocessing, runs the deep learning model, and generates a 3D representation of the floorplan.
- **View VR Output:** Once the 3D model is generated, the user can visualize it in a virtual reality environment. This allows interactive exploration of the building layout in an immersive manner.
- **Export 3D Model:** The system provides the option to export the generated 3D model in .glb format. This allows the user to use the model in other applications such as VR platforms, game engines, or architectural tools.

The use case diagram emphasizes the core functionalities available to the user and how the system supports these actions. It provides a high-level understanding of system capabilities, making it easier to identify user requirements and system behavior from a functional perspective.

The proposed VRGen system efficiently converts 2D architectural floorplans into realistic 3D VR walkthroughs by combining deep learning techniques with image processing and 3D reconstruction methods. The integration of Enhanced U-Net, GAN architecture, and post-processing ensures high-quality output, making the system suitable for real estate visualization and architectural design applications.

Chapter 6

Experimental Setup

6. Experimental Setup

6.1 Details of Database / Input Data to the System

The proposed VRGen system utilizes a publicly available dataset titled “Text to Floorplan using GANs” obtained from Kaggle. The dataset consists of architectural floor plan images that serve as the primary input for training the deep learning model.

The dataset contains a total of 4003 images, out of which 1000 images are used for training and 500 images are used for testing, while the remaining images support validation and augmentation processes. All input images represent 2D architectural floor plans including walls, room layouts, and structural boundaries.

Before feeding the data into the model, preprocessing is performed to ensure consistency and feature clarity. Each image is resized to a fixed resolution of 256×256 pixels and normalized. Additional preprocessing steps include grayscale conversion, contrast enhancement, and edge detection, which help emphasize architectural structures such as walls and room separations.

Since annotated heightmaps are not available, the system generates synthetic target heightmaps during training. These are created by combining grayscale intensity information with detected edges, ensuring that walls are represented with higher intensity values.

To improve generalization and address data availability, several data augmentation techniques are applied, including:

- Horizontal and vertical flipping
- Rotation (90°, 180°, 270°)
- Brightness and contrast adjustments

This preprocessing pipeline ensures that the model learns robust spatial and structural representations from diverse architectural layouts.

6.2 Performance Evaluation Parameters (Validation Metrics)

The performance of the proposed VRGen system is evaluated using multiple loss-based metrics during the training phase. These metrics are carefully selected to measure both the structural accuracy and visual quality of the generated heightmaps. Since the system is based on a GAN architecture, evaluation focuses on how well the generator produces realistic outputs and how effectively the discriminator distinguishes them from real data. Together, these metrics ensure that the generated results are not only visually convincing but also architecturally meaningful.

Primary Evaluation Metrics:

- **Generator Loss (G Loss):** The generator loss represents the overall performance of the generator in producing realistic and accurate heightmaps. It combines adversarial loss with additional losses such as L1, edge, and perceptual loss. During training, the generator loss stabilizes in the range of 21.7 to 24.6, indicating proper convergence and consistent learning behavior.
- **Discriminator Loss (D Loss):** The discriminator loss measures how well the discriminator can differentiate between real and generated outputs. As training progresses, this loss approaches 0.0000, suggesting that the generator becomes highly effective at producing realistic outputs that are difficult for the discriminator to distinguish.
- **L1 Loss (Reconstruction Loss):** L1 loss ensures pixel-wise similarity between the generated heightmap and the ground truth. It helps preserve spatial alignment and structural consistency. The observed range of 0.63 to 1.55 indicates that the model successfully reconstructs the key architectural features.
- **Edge Loss (Edge Preservation):** Edge loss focuses on preserving sharp boundaries such as walls, room partitions, and structural edges. Maintaining these edges is critical for

architectural accuracy. The loss values range between 0.87 and 1.57, demonstrating effective edge preservation in generated outputs.

- **Perceptual Loss:** Perceptual loss measures high-level feature similarity using deep neural network features (e.g., VGG). It ensures that the generated heightmaps maintain structural coherence beyond pixel-level similarity. Values ranging from 0.48 to 1.11 indicate improved visual and structural consistency.
- **Total Variation (TV) Loss:** TV loss promotes smoothness in the generated heightmaps by reducing noise and unwanted variations in non-edge regions. This helps in producing cleaner and more visually appealing outputs while maintaining important structural details.

6.3 Software and Hardware Setup

The VRGen system is implemented using Python and a combination of deep learning, image processing, and 3D visualization libraries. The setup is designed to ensure efficient training, accurate processing, and realistic output generation.

6.3.1 Software Configuration

The system is implemented using Python with deep learning and image processing libraries. The primary software components include:

1. Frameworks and Libraries:

- **PyTorch (Deep Learning):** Used to design, train, and optimize the conditional GAN model for floorplan-to-heightmap generation.
- **Torchvision (Data Handling):** Provides image transformations, preprocessing utilities, and dataset loading support for training.
- **OpenCV (Image Processing):** Used for edge detection, morphological operations, and enhancement of architectural features.
- **NumPy (Numerical Computation):** Handles array operations and numerical processing for image and heightmap data.
- **SciPy (Scientific Computing):** Supports advanced mathematical operations and image processing tasks such as filtering.
- **Pillow (PIL) (Image Handling):** Used for image loading, resizing, format conversion, and basic preprocessing.
- **Trimesh (3D Processing):** Converts generated heightmaps into 3D mesh representations for visualization.

- **PyRender (3D Visualization):** Enables rendering and visualization of generated 3D models in virtual environments.
- **Plotly (Visualization):** Used for interactive visualization and debugging of model outputs and results.
- **PyYAML (Configuration Management):** Handles reading and managing training configurations from YAML files.
- **TQDM (Progress Monitoring):** Provides real-time progress bars during model training and data processing.

2. Model Architecture:

The proposed system is built on a Generative Adversarial Network (GAN) framework specifically optimized for image-to-image translation tasks. This architecture enables the system to learn a direct mapping from input floorplan images to corresponding heightmaps. By combining a powerful generator and discriminator, the model ensures that the generated outputs are both structurally accurate and visually realistic. The use of supervised learning further improves performance by guiding the model with paired training data.

- **Conditional GAN (Pix2Pix-based):** The system utilizes a conditional GAN inspired by the Pix2Pix framework, where the generation process is conditioned on the input image. This means that the model learns the relationship between the floorplan and its corresponding heightmap rather than generating outputs randomly. The supervised nature of this approach allows the model to produce more consistent and meaningful results, preserving the layout and structural details of the input floorplan.
- **Enhanced U-Net Generator with Attention Mechanisms:** The generator is based on an enhanced U-Net architecture that follows an encoder–decoder structure. The encoder progressively extracts high-level features from the input image, while the decoder reconstructs the output heightmap. Skip connections are used to retain spatial information and prevent loss of fine details. Additionally, attention mechanisms are integrated into the network to focus on important architectural features such as walls, doors, and edges. This improves the model’s ability to capture both global layout and local structural details, resulting in more accurate and detailed heightmaps.
- **PatchGAN Discriminator:** The discriminator follows a PatchGAN architecture, which evaluates small patches of the image instead of the entire image at once. This localized evaluation helps the model focus on high-frequency details such as edges and textures, which are crucial for architectural accuracy. By distinguishing between real and generated

patches, the discriminator provides strong feedback to the generator, encouraging it to produce more realistic and structurally consistent outputs.

3. Training Configuration

- **Epochs (200):** Defines the total number of complete training iterations over the dataset for model learning.
- **Batch Size (12):** Specifies the number of images processed in one training step, balancing memory usage and training speed.
- **Learning Rate (0.0001):** Controls the step size for updating model weights to ensure stable convergence.
- **Optimizer – Adam ($\beta_1 = 0.5$, $\beta_2 = 0.999$):** Optimizes model parameters using adaptive moment estimation for efficient and stable training.

4. Loss Weights

- **L1 Loss (100):** Assigns high importance to pixel-wise reconstruction accuracy between generated and target heightmaps.
- **Edge Loss (50):** Emphasizes preservation of architectural boundaries such as walls and edges.
- **Perceptual Loss (10):** Enhances high-level structural similarity using feature-based comparison.
- **Total Variation Loss (0.1):** Promotes smoothness in generated outputs by reducing noise in non-edge regions.

5. Output Formats:

- **Heightmaps (PNG Format):** Stores generated grayscale height representations that capture depth information of the floor plan. These heightmaps are useful for further processing, analysis, and refinement of 3D structures. They act as an intermediate output that enhances realism in the reconstruction stage.
- **3D Models (GLB Format – VR Compatible):** Exports reconstructed 3D meshes in GLB format, which is lightweight and widely supported for web and Virtual Reality applications. This format allows seamless visualization, sharing, and integration into VR platforms. It enables users to interactively explore the generated models in an immersive environment.

6.3.2 Hardware Configuration

The system is designed to run on GPU-enabled environments to ensure efficient training and faster processing of deep learning models. GPU acceleration significantly reduces training time and improves overall performance.

- **Processing Unit:** NVIDIA GPU (CUDA-enabled) for parallel computation and faster model execution.
- **Recommended GPU Memory:**
 - **16 GB → Batch size 12:** Suitable for moderate training workloads with balanced performance.
 - **24 GB → Batch size 16:** Supports larger batch sizes and faster training for improved model efficiency.
- **RAM:** Minimum 8–16 GB to handle data preprocessing, model loading, and intermediate computations efficiently.
- **Storage:** Sufficient capacity is required to store datasets, trained models, and checkpoint files generated during training.
- **VR Headset (for Visualization):** Devices such as Oculus Quest or similar VR headsets are used to enable immersive exploration of the generated 3D models. These headsets allow users to interactively navigate through the virtual environment and experience the spatial layout in real time.
- **Device Used:** CUDA-based GPU for optimized deep learning operations and enhanced computational speed.

For inference, the system also supports CPU execution, allowing deployment in resource-constrained environments. Although CPU-based inference may be slower compared to GPU execution, it ensures accessibility and usability across a wider range of devices.

Chapter 7

Results and Discussion

7. Results and Discussion

7.1 Dataset and Training Configuration

The proposed VRGen system was evaluated using the Text-to-Floorplan using GANs dataset, which consists of a total of 4003 images. Out of these, 1000 images were used for training and 500 images were used for testing to validate the model’s performance. The model was trained under the following configuration:

- Input image size was fixed at 256×256 pixels
- Batch size was set to 12
- Training was performed for 200 epochs
- A CUDA-enabled GPU was used to accelerate computation

To ensure both structural accuracy and visual quality, a combination of multiple loss functions was used:

- **L1 Loss (100)** → for pixel-level reconstruction accuracy
- **Edge Loss (50)** → for preserving architectural boundaries

- **Perceptual Loss (10)** → for high-level feature similarity
- **Total Variation Loss (0.1)** → for smoothness and noise reduction

This multi-loss strategy helped the model learn both global layout and fine-grained details effectively.

7.2 Quantitative Results (Loss Analysis)

The performance of the model during training was evaluated using multiple loss metrics, which reflect how effectively the system learns both structural and visual features of the floor plans. These metrics provide insight into the convergence behavior of the model as well as the quality of generated heightmaps.

- **Generator Loss:** Stabilized between 21 and 24, indicating consistent convergence and stable learning throughout the training process.
- **Discriminator Loss:** Remained close to zero, suggesting that the generator produces outputs that are highly realistic and difficult to distinguish from real heightmaps.
- **L1 Loss:** Ranged between 0.58 and 1.54, demonstrating the model's strong ability to preserve spatial structure and layout of floor plans.
- **Edge Loss:** Varied between 0.87 and 1.57, confirming effective learning and preservation of important architectural features such as walls, doors, and boundaries.

In addition to these metrics, the gradual decay in the learning rate played an important role in improving training stability and overall performance.

- **Stable convergence:** Helped the model reach optimal performance smoothly
- **Reduced oscillations:** Minimized fluctuations during training
- **Better fine-tuning:** Allowed improved refinement in later epochs

7.3 Loss Behavior Analysis

The variation of L1 loss and edge loss across training epochs is illustrated in Graph 7.1. This graph provides important insight into how the model evolves during the training process and how its learning focus changes over time. In the initial stages of training, both L1 loss and edge loss remain relatively low because the model primarily learns basic structural patterns and overall layout of the floor plans. As training progresses, both losses gradually increase,

reflecting the model's shift towards capturing more complex and fine-grained architectural features.

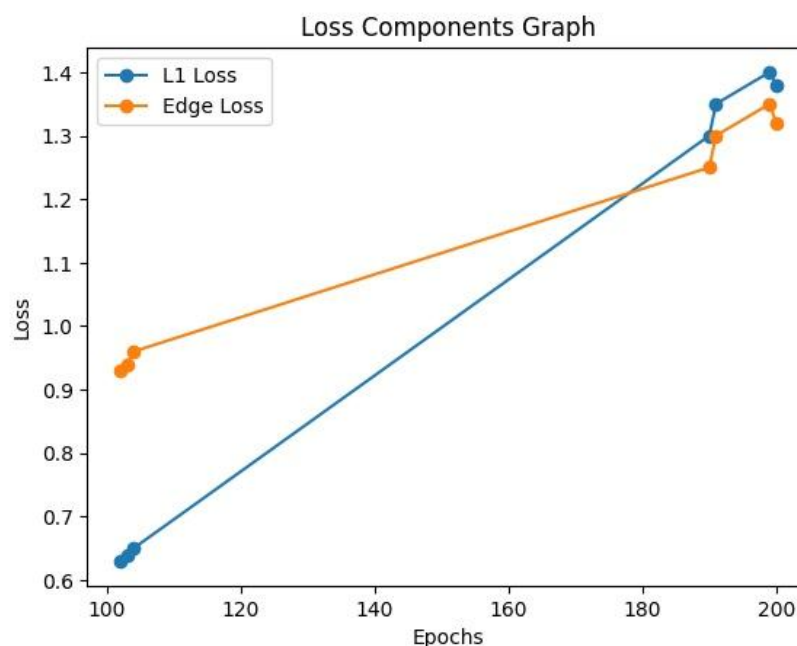
- **Initially:** Both L1 loss and edge loss remain relatively low
- **Later stages:** Both losses gradually increase as training progresses

This increasing trend does not indicate poor performance; instead, it shows that the model is learning more detailed and challenging aspects of the data. As the model improves, it focuses less on coarse structures and more on refining spatial relationships and edge continuity, which are essential for realistic 3D reconstruction.

- The model shifts from learning coarse structures to fine details
- Increase in L1 loss reflects better spatial refinement and layout consistency
- Increase in edge loss indicates improved capture of sharp and continuous boundaries

Overall, the graph demonstrates that the model is progressively enhancing its ability to represent architectural features with higher precision. It effectively balances structural preservation with edge enhancement, ensuring that both the overall layout and fine details are accurately maintained in the generated outputs.

- Focuses on fine-grained architectural details over time
- Maintains a balance between structure preservation and edge enhancement



Graph 7.1: Variation of L1 Loss and Edge Loss Across Training Epochs

7.4 Qualitative Results

The qualitative performance of the proposed system is illustrated in Figure 7.1, which presents the complete pipeline from input floor plan to the final 3D output. This figure helps in visually understanding how the system progressively transforms a 2D floor plan into a realistic 3D model. It highlights the effectiveness of each stage in improving the quality, clarity, and structural accuracy of the generated output.

The process begins with the generation of a raw prediction, where the model produces an initial heightmap directly from the input 2D floor plan. At this stage, the output captures the basic spatial layout but may still contain noise and less-defined boundaries. This intermediate result is then passed through the post-processing module, where several refinement techniques are applied to enhance the overall quality of the heightmap.

- **Raw Prediction:**
 - Initial heightmap generated from the 2D floor plan
 - Captures basic layout and structure
- **Refined Heightmap:** Post-processing significantly improves the output by:
 - Enhancing boundary clarity
 - Improving structural consistency
 - Reducing noise and unwanted artifacts

After refinement, the enhanced heightmap is used to generate the final 3D model. This stage converts the processed heightmap into a mesh representation, resulting in a visually realistic and structurally accurate model. The generated 3D output clearly reflects the architectural elements of the original floor plan, making it suitable for visualization and VR applications.

- **Final 3D Model:**
 - Provides realistic spatial representation
 - Ensures accurate wall continuity
 - Maintains proper room segmentation

A key factor contributing to this performance is the use of the conditional GAN (cGAN), which enables effective learning of the mapping between 2D floor plans and their corresponding depth representations. By incorporating both adversarial learning and reconstruction losses, the model is able to generate outputs that are not only structurally accurate but also visually realistic.

- **Role of cGAN:**
 - Generates depth information from 2D inputs
 - Enhances realism and quality of the final 3D model

Overall, the qualitative results demonstrate that the system successfully preserves important architectural features while producing visually coherent and immersive 3D representations.

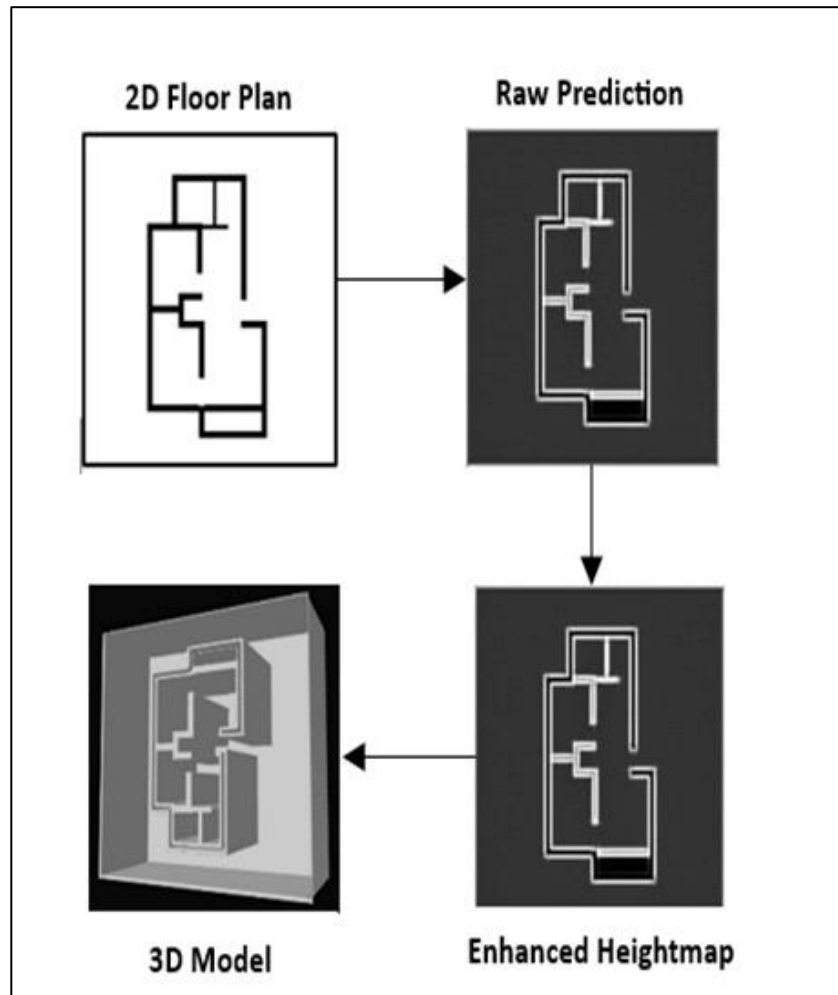


Figure 7.1 Pipeline for converting 2D floor plans into 3D Models

7.5 Comparison with Existing Systems

A comparison with existing approaches as shown in table 7.1 highlights the advantages of the proposed system.

Table 7.5.1: Comparative Advantage Over Existing Systems

Aspect	Existing Systems	Proposed VRGen System
Input & Output	Limited to 2D outputs or partial 3D	Converts 2D floorplans to complete 3D (.glb)
3D Reconstruction Capability	Partial or not fully implemented	Full end-to-end 3D reconstruction
Model Approach	CNN / GAN / rule-based (varies)	Conditional GAN (Pix2Pix-based)
Loss & Evaluation	Limited or not reported	Multi-loss with explicit evaluation
Output Quality (Structure + Edges)	Moderate, lacks detail in edges	High structural accuracy with sharp edges
Automation	Semi-automated	Fully automated pipeline
VR Integration	Not supported or limited	Fully VR-compatible walkthrough

A comparison with existing approaches based on key system aspects highlights the improvements offered by the proposed VRGen system. Most existing methods are limited in terms of end-to-end functionality and often focus on either 2D outputs or partial 3D reconstruction without full pipeline integration.

- **Input & Output:** Existing systems are typically restricted to 2D results or incomplete 3D outputs, whereas the proposed system generates complete 3D models from 2D floor plans.
- **3D Reconstruction:** Many approaches provide only partial reconstruction, while the proposed system supports full end-to-end 3D generation.
- **Model Approach:** Existing methods vary across different techniques, whereas the proposed system uses a consistent conditional GAN-based approach.
- **Loss & Evaluation:** Most systems lack detailed evaluation, while the proposed model includes explicit multi-loss optimization and numerical analysis.
- **Automation:** Existing solutions are often semi-automated, whereas the proposed system is fully automated.
- **VR Integration:** VR support is generally absent in existing works but is fully integrated in the proposed system.

Overall, the proposed VRGen system provides a more complete, automated, and practically applicable solution compared to existing approaches.

7.6 Overall Performance Evaluation

The overall performance of the proposed system demonstrates strong effectiveness across both quantitative and qualitative aspects. The loss-based evaluation confirms that the model achieves stable convergence and accurately learns spatial and structural features, while the visual results validate the system's ability to generate realistic and meaningful outputs. Together, these evaluations indicate that the system is capable of transforming 2D floor plans into high-quality 3D representations with good consistency and detail.

- **Quantitative performance:** Based on loss values, the model shows stable training behavior and reliable learning of structural patterns
- **Qualitative performance:** Visual outputs confirm realistic reconstruction and clear architectural representation

The strengths of the system are evident in its ability to maintain both global structure and fine details throughout the pipeline. The integration of deep learning with post-processing and 3D reconstruction techniques ensures that the generated outputs are not only accurate but also visually coherent and suitable for real-world applications.

- **Accurate structural reconstruction:** Preserves spatial layout and relationships between rooms
- **Sharp edge preservation:** Maintains clear boundaries for walls, doors, and partitions
- **Realistic 3D model generation:** Produces visually convincing and spatially accurate models
- **Effective system integration:** Combines deep learning, image processing, and 3D mesh generation into a unified workflow

Overall, the system achieves a good balance between accuracy, visual quality, and practical usability, making it a reliable solution for floor plan-based 3D reconstruction and VR visualization.

Chapter 8

Conclusion and Future Work

8. Conclusion and Future Work

8.1 Conclusion

This research presented VRGen, an end-to-end deep learning-based system for automatically converting 2D architectural floor plans into 3D models suitable for Virtual Reality walkthroughs. The system leverages a conditional GAN (Pix2Pix-based architecture) combined with an enhanced U-Net generator and attention mechanisms to effectively learn spatial and structural relationships between floor plan images and their corresponding heightmap representations. The integration of multiple loss functions, including L1, edge, perceptual, and total variation losses, ensures a balanced performance in terms of reconstruction accuracy, edge preservation, and visual consistency.

The experimental results demonstrate that the proposed approach successfully generates structurally coherent heightmaps with clear architectural boundaries and minimal noise. The post-processing and 3D mesh reconstruction pipeline further transforms these outputs into VR-compatible models in GLB format, enabling interactive and immersive visualization. This eliminates the need for manual 3D modeling, significantly reducing time, effort, and technical

expertise. The system proves to be highly effective for real estate pre-selling, architectural visualization, and immersive design presentations.

Overall, the VRGen system highlights the potential of deep learning in bridging the gap between 2D architectural designs and immersive 3D environments. It provides a scalable, efficient, and automated solution for next-generation real estate and architectural applications, enhancing user experience and decision-making through realistic virtual walkthroughs.

8.2 Future Work

- **Expansion of Dataset:** The performance of the system can be further improved by increasing the size and diversity of the training dataset. Including a wide range of architectural styles, layouts, and complexities will help the model generalize better to real-world scenarios. A more diverse dataset will reduce bias and improve robustness. This will enable the system to handle complex and irregular floor plans more effectively.
- **Semantic Element Detection:** Future work can incorporate semantic segmentation techniques to detect and classify architectural elements such as doors, windows, furniture, and room types. This will allow the system to generate more detailed and meaningful 3D models. It will enhance spatial understanding and realism. Such improvements will make the system more suitable for practical architectural and interior design applications.
- **Advanced Texturing and Materials:** Integrating texture synthesis and material mapping techniques can significantly enhance the visual quality of generated 3D models. This includes adding realistic wall textures, flooring, lighting effects, and material properties. Such enhancements will make the virtual environment more immersive and visually appealing. It will also improve user experience in VR-based walkthroughs.
- **Real-Time Processing Optimization:** Optimizing the model for faster inference is an important area for future work. Techniques such as model compression, pruning, and efficient architectures can be used to reduce processing time. This will enable near real-time generation of 3D models from 2D inputs. Faster processing will improve usability in practical applications where quick results are required.
- **Deployment on Lightweight Devices:** Future improvements can focus on adapting the system for deployment on mobile devices and standalone VR headsets. This will make the system more accessible to a wider range of users. Optimization for low-resource

environments will ensure smooth performance without high-end hardware. It will also support real-world deployment in commercial applications.

- **Multi-Modal Input Support:** The system can be extended to accept multiple input formats such as hand-drawn sketches, textual descriptions, or voice commands. This will increase flexibility and usability for different types of users. It will allow non-technical users to generate 3D models easily. Such integration will make the system more versatile and user-friendly.
- **Enhanced VR Interaction:** Future work can improve user interaction within the VR environment by adding advanced features such as navigation controls, annotations, and guided walkthroughs. This will allow users to explore spaces more intuitively. Interactive elements can enhance engagement and understanding of the layout. It will further strengthen the immersive experience provided by the system.

References

- [1] H. Huang, S. Liu, J. Liu, and P. Jin, “Learning to Predict 3D Meshes from a Single Image via Depth Consistency,” *Chinese Journal of Mechanical Engineering*, vol. 38, 2025.
- [2] M. Muller et al., “GenAICHI 2025: Generative AI and HCI at CHI 2025,” in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, Yokohama, Japan, 2025, pp. 1–10.
- [3] V. K. Reddy G. N., V. Prajwal, V. G. Reddy N., S. G. Srushti, and N. S., “Plan–MORPH: Conversion of 2D Floorplans to 3D Models,” *International Journal of Enhanced Research in Science, Technology & Engineering*, vol. 13, no. 12, pp. 37–41, Dec. 2024.
- [4] A. Cambeiro Barreiro, M. Trzeciakiewicz, A. Hilsmann, and P. Eisert, “Automatic Reconstruction of Semantic 3D Models from 2D Floor Plans,” *arXiv preprint arXiv:2306.01642*, 2023.
- [5] K. Sahray, A. S. Sukereman, S. H. Rosman, and F. N. H. Jaafar, “The implementation of Virtual Reality (VR) technology in real estate industry,” *Planning Malaysia: Journal of the Malaysian Institute of Planners*, vol. 21, no. 3, pp. 255–265, 2023.
- [6] M. J. Buehler, “Generating 3D architected nature-inspired materials and granular media using diffusion models based on language cues,” *Oxford Open Materials Science*, vol. 2, no. 1, 2022.

- [7] S. Park and H. Kim, “3DPlanNet: Generating 3D models from 2D floor plan images using ensemble methods,” *Electronics*, vol. 10, no. 22, p. 2729, 2021.
- [8] R. G. Kippers, M. Koeva, M. van Keulen, and S. J. Oude Elberink, “Automatic 3D building model generation using deep learning methods based on CityJSON and 2D floor plans,” in *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. XLVI-4/W4, 2021, pp. 49–56.
- [9] Y.-K. Juan, H.-H. Chen, and H.-Y. Chi, “Developing and evaluating a virtual reality-based navigation system for pre-sale housing sales,” *Applied Sciences*, vol. 8, no. 6, p. 952, 2018.
- [10] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1125–1134.
- [11] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual losses for real-time style transfer and super-resolution,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 694–711.
- [12] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, vol. 9351, Springer, 2015, pp. 234–241.
- [13] U. Gurevich and R. Sacks, “Integrated Virtual Reality and Discrete Event Simulation Methods for Production System Research in Construction,” in *Proc. 13th Int. Conf. Construction Applications of Virtual Reality*, London, U.K., 2013.
- [14] L. I. Rudin, S. Osher, and E. Fatemi, “Nonlinear total variation based noise removal algorithms,” *Physica D: Nonlinear Phenomena*, vol. 60, no. 1–4, pp. 259–268, Nov. 1992.
- [15] I. Sobel and G. Feldman, “A 3×3 isotropic gradient operator for image processing,” presented at the Stanford Artificial Intelligence Project (SAIL), 1968 (referenced in R. O. Duda and P. E. Hart, *Pattern Classification and Scene Analysis*, 1975).

Appendix

Enhanced Inference Pipeline for Floorplan-to-3D Conversion

This script implements the inference stage of the proposed system, converting a 2D floorplan image into a 3D mesh model using a trained Enhanced U-Net generator. It loads a pretrained checkpoint, preprocesses the input image, and generates a heightmap along with edge predictions. The code further applies post-processing techniques to clean the heightmap and improve structural clarity. An additional enhancement step ensures clear distinction between floor and wall regions for better 3D representation. Finally, the processed heightmap is converted into a 3D mesh with adjustable height scaling, and intermediate outputs are saved for debugging and analysis.

Code:

```
import os
import torch
from PIL import Image
import torchvision.transforms as T
import numpy as np
from models_enhanced import EnhancedUNetGenerator
from utils import heightmap_to_mesh
from postprocess import clean_heightmap

def load_checkpoint(net, ckpt_path, device):
    """Load checkpoint - handles both old and new format"""
    checkpoint = torch.load(ckpt_path, map_location=device)

    # Handle different checkpoint formats
    if isinstance(checkpoint, dict) and 'model_state_dict' in checkpoint:
        net.load_state_dict(checkpoint['model_state_dict'])
        print(f"Loaded checkpoint from epoch {checkpoint.get('epoch', 'unknown')}")
    else:
        net.load_state_dict(checkpoint)
    net.to(device)
    net.eval()
    return net

def enhance_heightmap_for_3d(heightmap, floor_level=0.1, wall_level=0.9,
threshold=0.5):
    """
    Enhance heightmap to ensure clear floor-wall distinction for 3D meshing
    """
```



```

# Create binary mask for walls
wall_mask = heightmap > threshold
# Set floors to low level and walls to high level
enhanced = heightmap.copy()
enhanced[~wall_mask] = floor_level # Floors
enhanced[wall_mask] = wall_level # Walls
return enhanced
def infer_one(floorplan_path, cfg_path="config.yaml", ckpt=None,
              out_mesh="out_initial_mesh.glb", apply_postprocess=True):
    """
    Run inference on a single floorplan
    Args:
        floorplan_path: path to input floorplan image
        cfg_path: config file path
        ckpt: checkpoint path
        out_mesh: output mesh path
        apply_postprocess: whether to apply post-processing
    """
    device = torch.device("cpu") # Force CPU for compatibility
    # Defaults
    ckpt = ckpt or "./checkpoints/netG_best.pth"
    if not os.path.exists(ckpt):
        ckpt = "./checkpoints/netG_final.pth"
    img_size = 256
    print(f'Loading model from: {ckpt}')
    # Load model
    netG = EnhancedUNetGenerator(in_channels=3, out_channels=1)
    netG = load_checkpoint(netG, ckpt, device)
    # Preprocess input
    img = Image.open(floorplan_path).convert('RGB')
    original_size = img.size
    img_resized = img.resize((img_size, img_size), Image.LANCZOS)
    transform = T.Compose([T.ToTensor()])
    inp = transform(img_resized).unsqueeze(0).to(device)
    print("Running inference...")
    with torch.no_grad():
        pred, pred_edges = netG(inp) # Get both outputs
        pred = (pred + 1) / 2.0 # Scale from [-1,1] to [0,1]
        pred_np = pred.squeeze(0).squeeze(0).cpu().numpy()
    # Save raw prediction
    png_out = out_mesh.replace(".glb", "_raw.png")
    os.makedirs(os.path.dirname(png_out) or ".", exist_ok=True)
    Image.fromarray((pred_np * 255).astype('uint8')).save(png_out)
    print(f'Saved raw prediction: {png_out}')
    # Apply post-processing
    if apply_postprocess:

```

```

print("Applying post-processing...")
cleaned_path, pred_np = clean_heightmap(
    png_out,
    floor_img_path=floorplan_path,
    output_path=out_mesh.replace(".glb", ".png")
)
print(f"Saved cleaned heightmap: {cleaned_path}")
else:
    # Just save without post-processing
    cleaned_path = out_mesh.replace(".glb", ".png")
    Image.fromarray((pred_np * 255).astype('uint8')).save(cleaned_path)
# Enhance heightmap for 3D visibility (ensure clear floor-wall distinction)
print("Enhancing heightmap for 3D visibility...")
enhanced_pred_np = enhance_heightmap_for_3d(
    pred_np,
    floor_level=0.1, # Low floor level
    wall_level=0.9, # High wall level
    threshold=0.4    # Adjust based on your data
)
# Convert to mesh with increased Z-scale for visible walls
if not out_mesh.endswith(".glb"):
    out_mesh = out_mesh + ".glb"
print("Converting to 3D mesh...")
print(f"Heightmap stats - Min: {enhanced_pred_np.min():.3f}, Max:
{enhanced_pred_np.max():.3f}, Mean:
{enhanced_pred_np.mean():.3f}")
# Increased Z-scale to make walls clearly visible
mesh_out = heightmap_to_mesh(
    enhanced_pred_np,
    out_path=out_mesh,
    scale_xy=1.0,
    scale_z=10.0 # ← Increased from 2.0 to 10.0
)
print(f"Saved mesh: {mesh_out}")
# Also save enhanced heightmap for debugging
enhanced_out = out_mesh.replace(".glb", "_enhanced.png")
Image.fromarray((enhanced_pred_np * 255).astype('uint8')).save(enhanced_out)
print(f"Saved enhanced heightmap: {enhanced_out}")
# Also save edge prediction for debugging
edge_out = out_mesh.replace(".glb", "_edges.png")
edge_np = pred_edges.squeeze(0).squeeze(0).cpu().numpy()
Image.fromarray((edge_np * 255).astype('uint8')).save(edge_out)
print(f"Saved edge prediction: {edge_out}")
return mesh_out
if __name__ == "__main__":
    import argparse

```

```

p = argparse.ArgumentParser(description="Enhanced floorplan to 3D inference")
p.add_argument("image", help="Input floorplan image path")
p.add_argument("--ckpt", default="./checkpoints/netG_best.pth",
               help="Model checkpoint path")
p.add_argument("--out", default="tmp_pipeline/initial_mesh.glb",
               help="Output mesh path")
p.add_argument("--no-postprocess", action="store_true",
               help="Skip post-processing")
p.add_argument("--scale-z", type=float, default=10.0, # ← Added option to adjust
               Z-scale
               help="Z-axis scaling factor for wall height")
args = p.parse_args()
# Modify the function to use the command-line scale_z
def infer_one_with_scale(floorplan_path, cfg_path="config.yaml", ckpt=None,
                        out_mesh="out_initial_mesh.glb", apply_postprocess=True,
                        scale_z=10.0):
    device = torch.device("cpu") # Force CPU for compatibility
    # Defaults
    ckpt = ckpt or "./checkpoints/netG_best.pth"
    if not os.path.exists(ckpt):
        ckpt = "./checkpoints/netG_final.pth"
    img_size = 256
    print(f'Loading model from: {ckpt}')
    # Load model
    netG = EnhancedUNetGenerator(in_channels=3, out_channels=1)
    netG = load_checkpoint(netG, ckpt, device)
    # Preprocess input
    img = Image.open(floorplan_path).convert('RGB')
    original_size = img.size
    img_resized = img.resize((img_size, img_size), Image.LANCZOS)
    transform = T.Compose([T.ToTensor()])
    inp = transform(img_resized).unsqueeze(0).to(device)
    print("Running inference...")
    with torch.no_grad():
        pred, pred_edges = netG(inp) # Get both outputs
        pred = (pred + 1) / 2.0 # Scale from [-1,1] to [0,1]
        pred_np = pred.squeeze(0).squeeze(0).cpu().numpy()
    # Save raw prediction
    png_out = out_mesh.replace(".glb", "_raw.png")
    os.makedirs(os.path.dirname(png_out) or ".", exist_ok=True)
    Image.fromarray((pred_np * 255).astype('uint8')).save(png_out)
    print(f'Saved raw prediction: {png_out}')
    # Apply post-processing
    if apply_postprocess:
        print("Applying post-processing...")
        cleaned_path, pred_np = clean_heightmap(

```

```

        png_out,
        floor_img_path=floorplan_path,
        output_path=out_mesh.replace(".glb", ".png")
    )
    print(f"Saved cleaned heightmap: {cleaned_path}")
else:
    # Just save without post-processing
    cleaned_path = out_mesh.replace(".glb", ".png")
    Image.fromarray((pred_np * 255).astype('uint8')).save(cleaned_path)
# Enhance heightmap for 3D visibility (ensure clear floor-wall distinction)
print("Enhancing heightmap for 3D visibility...")
enhanced_pred_np = enhance_heightmap_for_3d(
    pred_np,
    floor_level=0.1, # Low floor level
    wall_level=0.9, # High wall level
    threshold=0.4    # Adjust based on your data
)
# Convert to mesh with increased Z-scale for visible walls
if not out_mesh.endswith(".glb"):
    out_mesh = out_mesh + ".glb"
print("Converting to 3D mesh...")
print(f"Heightmap stats - Min: {enhanced_pred_np.min():.3f}, Max:
{enhanced_pred_np.max():.3f}, Mean:
{enhanced_pred_np.mean():.3f}")
# Use the command-line scale_z
mesh_out = heightmap_to_mesh(
    enhanced_pred_np,
    out_path=out_mesh,
    scale_xy=1.0,
    scale_z=scale_z # ← Use command-line argument
)
print(f"Saved mesh: {mesh_out}")
# Also save enhanced heightmap for debugging
enhanced_out = out_mesh.replace(".glb", "_enhanced.png")
Image.fromarray((enhanced_pred_np * 255).astype('uint8')).save(enhanced_out)
print(f"Saved enhanced heightmap: {enhanced_out}")
# Also save edge prediction for debugging
edge_out = out_mesh.replace(".glb", "_edges.png")
edge_np = pred_edges.squeeze(0).squeeze(0).cpu().numpy()
Image.fromarray((edge_np * 255).astype('uint8')).save(edge_out)
print(f"Saved edge prediction: {edge_out}")
return mesh_out
infer_one_with_scale(
    args.image,
    ckpt=args.ckpt,
    out_mesh=args.out)

```

Publication

Welcome Vijesh Mundokalam [Sign out](#)

Controller General of Patents, Designs & Trade Marks



सत्यमेव जयते
G.A.R.6
[See Rule 22(1)]
RECEIPT



Docket No 32207

Date/Time 2026/04/22 13:30:36

To
Vijesh Mundokalam

UserId: Vijesh26

Flat No-1103,11th Floor, B Wing - Saki,
Puranik Tokyo Bay Phase 1, Opp:Haware
City, G.B.Road, Kasarvadavali- 400615,
Thane(W). Maharashtra

CBR Detail:

Sr. No.	App. Number	Ref. No./Application No.	Amount Paid	C.B.R. No.	Form Name	Remarks
1	202621051136	TEMP/E-1/56141/2026-MUM	1600	17500	FORM 1	"VRGen: Deep Learning Based VR Property Walkthroughs Using 2D Plans"

TransactionID	Payment Mode	Challan Identification Number	Amount Paid	Head of A/C No
N-0001932016	Online Bank Transfer	2204260031519	1600.00	1475001020000001

Total Amount : ₹ 1600.00

Amount in Words: Rupees One Thousand Six Hundred Only

Received from Vijesh Mundokalam the sum of ₹ 1600.00 on account of Payment of fee for above mentioned Application/Forms.

* This is a computer generated receipt, hence no signature required.

[Print](#)

[Home](#)

[About Us](#)

[Contact Us](#)

4/21/26, 9:29 PM

APSIT College Mail - VRGen: Deep Learning Based VR Property Walkthroughs Using 2D Plans



YASH PENKAR <yashpenkar106@apsit.edu.in>

VRGen: Deep Learning Based VR Property Walkthroughs Using 2D Plans

1 message

scopus scie <icsiehybrid@gmail.com>
To: yashpenkar106@apsit.edu.in

Fri, Apr 10, 2026 at 8:00 PM

Dear Author/s,

We are happy to inform you that your paper, submitted for the ICSIE 2026 conference has been **Accepted** based on the recommendations provided by the Technical Review Committee. By this mail you are requested to proceed with Registration for the Conference. Most notable is that the Conference must be registered on or before **April 23rd, 2026** from the date of acceptance.

www.icsie.co.in

Kindly fill the **Registration form(NOTE-JOURNAL DETAILS, PAYMENT DETAILS), Certificate form** which is attached with the mail and it should reach us on above mentioned days.

Instructions to fill the forms:

- Ensure to send **payment screenshots** and send all the details once the payment has been done to the account.
- All the above completed details should be mailed to icsiehybrid@gmail.com
- Please send a soft copy of the RESEARCH PAPER in word format only.

We reserve the right to reject your paper if the registration is not done within the above said number of days.

Paper id: ICSIEDR2602297

ICSIE 2026
www.icsie.co.in
9952936516

2 attachments

 **Certificates form - ICSIE 2026 (1).xls**
32K

 **Registration Instructions & Declaration form - 17TH ICSIE 2026 (1).doc**
934K

<https://mail.google.com/mail/u/4/?ik=c0e156413b&view=pt&search=all&permthid=thread-f:1862094204572710283&simpl=msg-f:1862094204572...> 1/1

VRGen: Deep Learning Based VR Property Walkthroughs Using 2D Plans

Prof. Vijesh Mundokalam, Yash Penkar, Prarthana Patil, Rutuja Pawar, and Dhruv Sawant

Computer Science Engineering (AI & ML), A. P. Shah Institute of Technology, Thane 400615,
Maharashtra, India

Abstract. In real estate and interior design visualization, and also in pre-selling systems, the conversion of architectural floor plans to three-dimensional representations is essential. The common workflow of modeling depends on traditional pipelines and tools such as CAD and Blender, which is timeconsuming and cannot be easily because of the necessity to recruit the services of professionals and re-work on any single change of design. This study suggests the use of a deep learning solution to create 3D models considering 2D floor plans in an automated manner using conditional Generative Adversarial Networks. The proposed scheme achieves spatial and structural characteristics using the basis of floor plan visualization to generate geometrically precise 3D data, which is engaged in interactive visualization in 3D. The findings of the experiment are the evidence of the viability of the approach as the demonstration of the possibility to utilize the approach to the larger datasets and more complicated architecture structures. The results determine the usefulness of deep learning to help in an expandable and immersive visualization of real estate and virtual reality solutions.

Keywords: Architectural Visualization · Floor Plan Analysis · 3D Reconstruction · Virtual Reality · Deep Learning

1 Introduction

Architectural visualization has become vital in today’s real estate development, interior design, and construction process, since two-dimensional (2D) floor plans are typically used to communicate the spatial relationships of an architectural system or layout. Floor plans do not convey a real understanding of spatial depth, scale, and the user experience associated with that space. Clearly, this can be a problem for those reviewing designs during early design reviews or during preselling, due to the difficulty in evaluating areas that are not yet built. As a result, there is considerable interest in the use of three-dimensional (3D) and immersive visualization techniques, because these techniques help people to more accurately visualize their spatial experiences and create a more engaged interaction between the user and the space [10], [11]. The ideal design scheme would enable stakeholders to easily navigate architectural representations and compare different design versions. Creating 3D models from 2D images has been recognized as time-consuming, expensive, and reliant upon specialist skilled operators; these constraints restrict scalability and require extensive time to adapt quickly when changes occur. The development of tools to produce 3D models via automated or semiautomated processes from architectural drawings has been addressed in numerous previous studies. Recent investigations have demonstrated that machine learning techniques provide the opportunity to extract structural patterns and spatial relationships from a floor plan and achieve partial reconstruction of the interior environment [1], [2]. Geometric consistency, semantic understanding, and visual fidelity are aspects of the same issue, with several additional articles describing ongoing research focused in this direction [3], [4]. Current techniques for developing interactive three-dimensional walkthroughs from two-dimensional floor plans have many limitations because they require complex workflows, intermediate representations, or additional manual refinement, which makes them impractical for deployment. Furthermore, other than those related to reconstructing accurately, most methods do not account for usability,

interactivity, or support for immersive environments. The fact that there are few lightweight, end-to-end methods for converting two-dimensional floor plans into interactive, three-dimensional walkthroughs indicates a large gap in the current body of research in this area. From a practical viewpoint, this affects real estate pre-selling, architectural design collaborations, and participatory planning, all of which require an understanding of space that is both easy to grasp and intuitive [10]. From an academic point of view, this shows that one of the major unresolved issues in architectural computing is how to effectively bridge the gap between the automated reconstruction of spaces and the immersive visualization of those same spaces in a scalable and accessible manner. This research aims to fill an existing gap by exploring a method for creating 3D interactive architectural models automatically, using 2D floor plans as the starting point.

2 Literature Survey

The challenge of transforming 2D architectural information into 3D spaces has plagued architects, developers, and builders for decades through the use of computers to visually communicate their designs. Floor plans are still the simplest and most widely used means of conveying spatial relationships, but because of their abstract nature they do not adequately convey spatial relationships for anyone who is not an expert in the field of architecture [10], [11]. Owing to this constraint, extensive research has focused on automating the reconstruction process, developing learning-based interpretations of architectural plans, and creating immersive visualization tools that enable individuals to comprehend and interact with designs prior to their construction [1],[4]. The early approaches to create three-dimensional models of buildings using rule-based systems and procedural modeling were limited. The models were based on certain assumptions regarding wall thickness, connectivity of rooms, and logic of construction, which could not withstand discrepancies of drafting styles and realities of real-world situations [5],[13]. Although earlier methods demonstrated promising results in a controlled experimental environment, they were not always robust and scalable when tested on different architectural standards, thus emphasizing the need for more data-driven approaches. However, as more mature machine learning technologies began to emerge, there has been progressive emphasis on data-driven methods that can learn and extract both structural as well as semantic data from architectural drawings. The possibility of reconstructing residential interior space from a single floor plan using neural networks, but their solution required pre-defined assumptions about layout organization [2]. In a related study, Park and Kim introduced 3DPlanNet, an ensemble-based framework designed to improve geometric consistency in 3D reconstruction from 2D floor plans [3]. To address scalability at an urban level, Kippers proposed a deep learning framework that integrates 2D floor plans with City JSON to generate large-scale 3D building models suitable for geographic information systems [1]. Although the method achieved accurate exterior and structural reconstruction, its focus remained largely on building envelopes rather than interior navigability and user-centered exploration. In parallel, generative learning models have gained popularity for architectural data synthesis. Conditional Generative Adversarial Networks (cGANs) demonstrated strong performance in image-to-image translation tasks, particularly following the work on paired adversarial learning [6]. Subsequent studies applied GAN-based approaches to architectural layout and floor plan generation, showing that such models can learn architectural priors and spatial distributions [7],[8]. However, the above attempts mostly targeted the creation of 2D layouts or intermediate forms and did not extend to 3D modeling or immersive applications. Work related to 3D output from 2D inputs has branched into the use of voxel-based and mesh-based 3D modeling. It has recently been shown that adding depth consistency constraints can enhance the geometric coherence in predicting 3D meshes from single images [16]. Despite these progresses transferring such knowledge in architectural environments, spatial semantics, room interconnectivity and the usability itself are new issues. In the area of VIS (visualization

& user experience) such immersive technologies as virtual reality (VR) becomes increasingly popular in architectural and real estate usages. Studied a VR-based navigation system for pre-sale housing and concluded that immersive walkthroughs significantly improved user understanding and engagement compared to traditional visualization media [4]. Industry focused studies further confirm the role of VR in reducing uncertainty and preselling decision acceleration [10],[11]. However, these systems rely on manually created 3D assets, which again brought scalability issues that automation seeks to overcome. Even though some works try to bring automation into reconstruction by means of immersive visualization. There are, however, some limitations to applications. Primarily, most of the applications involve some kind of preprocessing, labeling, and revision using external modeling applications not available in real-time applications. Besides, usability and interaction quality are frequently treated as secondary concerns. Sutcliffe and Deol Kaur emphasized the fact that by looking at cognitive load and interaction effectiveness in VR environments, highlighting an often-forgotten dimension in reconstruction-based studies [15]. The existing studies indicate the importance of knowledge-based techniques for the effective projection of spatial knowledge from a two-dimensional floor plan, the promising capabilities of generative models for the synthesis of visual representation, and the effectiveness of virtual reality for the enhancement of spatial knowledge [1],[2],[3]. However, these research areas are typically investigated independently. The need for the design of lean, end-to-end solutions integrating automated reconstruction techniques with immersive visualization within the context of architecture remains a challenge. The paper introduces an automated system for the creation of 3D environments for the purpose of immersive visualization in the context of architecture and real estate.

3 System Architecture and Methodology

The proposed system, VRGen, brings to the fore an end-to-end deep learning solution for the automatic generation of 3D property walkthroughs from 2D architectural floor plans. The proposed system does not require 3D modeling software such as CAD or Blender. This is accomplished by employing a conditional Generative Adversarial Network (cGAN), which generates realistic VR walkthroughs with minimal human intervention by learning to map 2D floor plan representations to corresponding 3D structural form. Automation also helps in reducing the time, cost, and expertise required in the process of architectural visualization. Such processes will also further enhance rapid design iteration and increase access during early-stage decision-making to nontechnical audiences. The proposed system will be capable of managing the complicated architectural designs and reduce human interference to both bring efficiency and precision. It produces 3D VR walkthroughs of high quality by seamlessly combining data preprocessing, model inference and output refinement to produce the final results. To further appreciate the general layout and the workflow of VRGen, Figure 3.1 presents the entire system including the key components, data flow as well as interaction between modules. The representation offers a graphical display of the mailing procedure of input floor plans. This value also draws attention to preprocessing procedures like edge location and normalization which help to facilitate feature extraction. It provides evidence of how numerous intermediation representations such as heightmaps and structural cues can be used to obtain a precise 3D reconstruction, which can be then visualized and results in the creation of an entirely navigable 3D VR world.

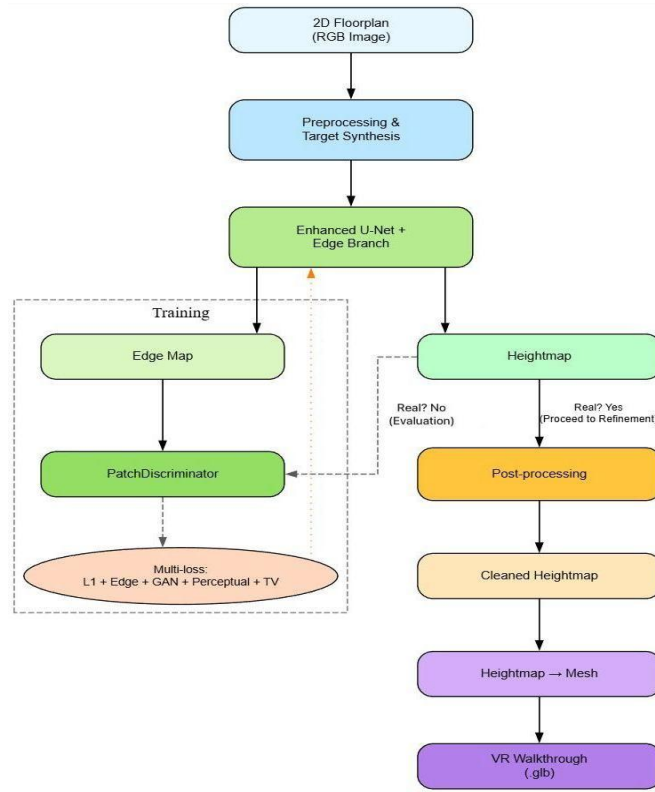


Figure 3.1: Overall System Architecture of VRGen

3.1 Input Data Pre-processing and Preparation of Dataset

Processing of inputs and data preparation are the foundation for the proposed system, which takes images with a two-dimensional view of the architectural floor plan. Figure 3.1 illustrates how the input floorplans are normalized and prepared for downstream modules. The images are then resized and normalized into a fixed resolution for uniformity during training. Along with enhancing the general architectural information of walls and the border of the room, there are certain input processing methods used in the proposed work, including grayscale conversion, improvement in contrast, and edge detection. As the amount of the annotated heightmap data is limited for the given task, the proposed system offers capability for automatic generation of the synthesized heightmap data directly from the images of floor plans by learning to estimate the corresponding heightmap representation during the course of the training. Some data augmentation techniques are used in the proposed system for introducing diversity into the dataset through methods involving uniform rotation, horizontal and vertical flipping, and brightness and contrast adjustment for the improvement in generalization even with small data.

3.2 Construction of Heightmaps Using Conditional GAN

The process of generating the heightmaps is performed using a conditional Generative Adversarial Network, which is a Pix2Pix architecture. In Figure 3.1, the flow from the preprocessed images through the enhanced U-Net and edge branch module is shown. The conditional GAN is capable of directly mapping the input image of the floor plan to the heightmap representation. The generator network uses an improved U-Net architecture, which utilizes the decoder and encoder layers along with the skip connections to focus on the details of the spatial representation across multiple feature scales. The attention techniques are also integrated into the network to highlight the

architectural details such as the walls, doors, and boundaries of the rooms. The network uses a PatchGAN discriminator to ensure that the generated heightmaps are realistic.

3.3 Heightmap Generation using of Deep Learning

In order to make the prediction of the heightmap of the architectures to be implemented as accurate as possible, there is a clear requirement for the use of the complex training process with the high value of the architectural loss function. In the case of the system at hand, the system uses the adversarial loss function as well as the L1 reconstruction loss function. Besides the two loss functions discussed above, for the specific use of attempting to conserve the boundaries of the architectural walls, the system uses the edge preservation loss function. In order to allow for a smooth transition for the nonedge areas for the optimal prediction of the architectural heightmaps, the system uses the perceptual as well as the total variation loss functions. Figure 3.1 highlights the interactions between these loss functions and the deep learning modules.

3.4 Post-processing and Architectural Detailing

After the prediction of the heightmap, a post-processing procedure is applied to enhance the quality of the predicted heightmap and extract better architectural features. The post processing technique involves noise removal with edge preservation, followed by wall enhancement to depict appropriate vertical separation, and finally, floor leveling to guarantee consistent height representation. Morphological operations are used to close the gaps and reduce the generation of artifacts; edges are sharpened to show sharp edges in architecture. The finalisation transformation is presented in Figure 3.1, describing the steps required to prepare the cleaned heightmap for a 3D mesh conversion.

3.5 3D Mesh Generation and VR Visualization

The resulting heightmap mesh can be used to reconstruct 3D walls and floor by simply scaling design z values. The mesh generation process supports scalable adjustment of wall height and dimensionality. The last module creates a VR walkthrough and transforms the fine-tuned heightmaps into a 3D mesh, as seen in Figure 3.1. Finally, the three-dimensional mesh is exported as a GLB file, enabling efficient visualization in virtual reality and web platforms without requiring the use of modeling tools for exploration and visualization. This allows the user, developer, or even a potential buyer to visualize a created architectural design through a virtual reality walkthrough. Lastly, Figure 3.1 shows how the cleaned heightmaps are transformed into a comprehensive 3D mesh, emphasizing the extrusion and refinement of walls, floors, and architectural elements.

4 Methodology

4.1 System Overview

In the study, the approach assumes a learning-based methodology framework that is aimed at automatically converting two-dimensional architectural floor planning into interactive three dimensional-environment to support the process of immersive visualization [1],[2]. The proposed system uses automated pipeline that involves floor plan image processing, learning based generation of heightmap, geometric reconstruction and deployment as applications to the virtual reality walkthroughs [3],[4]. The methodology framework makes architecture scalable, usable and consistent with little or no human intervention and maintenance of spatial accuracy [5]. The proposed architecture is designed and has been tested against its own research environment with a particular pipeline meant to train and test. The architecture is also

light and ready to deploy such that it can be visualized in both web and virtual reality environments [10],[11].

4.2 Dataset Preparation and Preprocessing

The architectural designs obtained from Kaggle [8]. The 2D images, which contain the wall structure designs, are pre-processed to retain a fixed spatial resolution. In addition, the normalization process is carried out on the entire dataset to ensure data uniformity throughout the training process. The preprocessing step converts images to grayscale, improves contrast, and extracts edges to highlight structural features, such as walls and room separations [6]. Uniform rotations, horizontal and vertical flipping, and controlled brightness and contrast changes are some of the data augmentation techniques used to reduce data scarcity and enhance generalization [2],[7]. By taking these actions, the most essential architectural semantic information is preserved while the model's robustness is increased.

4.3 Generator Architecture with Attention Mechanism

For the synthesis of the heightmaps, the proposed framework employs a modified encoder-decoder network with skip connections to preserve spatial details at high resolution. At the point where the skip connections meet, attention-based gates are introduced to enhance feature extraction at the decoding stage of the network [16]. These components help to enable the selective removal of uninformative features, thus enabling the reconstruction of key structural geometries, including wall parts and boundary interfaces. Let x represent the input feature map from the encoder feature map and let g_g represent the gating vector from the decoder. The attention coefficient ψ is measured using the following expression [16].

$$\psi = \sigma(BN(W_g g_g) + BN(W_x x))$$

where W_g and W_x characterize 1×1 convolutional kernels, while BN denotes batch normalization. The term σ represents the sigmoid activation. The final modulated output, which is refined feature map $\tilde{x}$ is calculated using element-wise multiplication [16],

$$\tilde{x} = x \odot \psi$$

This attentional framework enables the generator to prioritize salient architectural characteristics throughout the reconstruction stage, improving the spatial coherence of the heightmaps that were synthesized [3], [15].

4.4 Loss Functions and Training Objective

Perceptual quality, edge preservation, and structural precision must all be balanced when training the generator. Learning is guided by multiple complementary loss functions [6], [15], [16]. An L1 loss, defined as follows, is used to impose the reconstruction objective.

$$\mathcal{L}_{L1} = \|y - \hat{y}\|_1$$

Where y represents the actual heightmap and $\hat{y}$ the predicted output [6]. To keep the architectural edges, an edge-aware loss is calculated using Sobel operators S_x and S_y .

$$E(I) = \sqrt{(I * S_x)^2 + (I * S_y)^2 + \epsilon}$$

and the corresponding edge loss is

$$\mathcal{L}_{edge} = \| E(\hat{y}) - E(y) \|_1$$

Total variation regularization promotes spatial smoothness[18],

$$\mathcal{L}_{TV} = \sum_{i,j} ((\hat{y}_{i+1,j} - \hat{y}_{i,j})^2 + (\hat{y}_{i,j+1} - \hat{y}_{i,j})^2)$$

A pretrained convolutional network is used to compute a feature-based perceptual loss in order to enhance perceptual consistency,

$$\mathcal{L}_{perc} = \| \phi(\hat{y}^{(3)}) - \phi(y^{(3)}) \|_1$$

where $\phi(\cdot)$ signifies extracted feature maps and $(\cdot)^{(3)}$ stands for channel replication [19]. A discriminator that works on local patches gives adversarial supervision [6]. Expression for the adversarial loss is as follows,

$$\mathcal{L}_{GAN} = \mathbb{E}[\log D(x, y)] + \mathbb{E}[\log (1 - D(x, \hat{y}))]$$

The total generator loss is defined as,

$$\mathcal{L}_G = \mathcal{L}_{GAN} + \lambda_{L1}\mathcal{L}_{L1} + \lambda_{edge}\mathcal{L}_{edge} + \lambda_{perc}\mathcal{L}_{perc} + \lambda_{TV}\mathcal{L}_{TV}$$

4.5 Synthetic Heightmap Target Generation

During training, we create synthetic supervision because there are no annotated heightmaps. A smoothed grayscale image B is combined with a thickened edge map E to create a combined representation,

$$H_{synth} = \alpha B + (1 - \alpha)E$$

where $\alpha=0.3$. The resulting map is normalized as

$$H_{norm} = \frac{H_{synth} - \min(H_{synth})}{\max(H_{synth}) - \min(H_{synth}) + \epsilon}$$

This formulation provides stable training targets while preserving architectural structure.

4.6 3D Mesh Reconstruction and Visualization

Extrusion of height values along the vertical axis transforms the normalized heightmap into a volumetric representation. A binary voxel grid $V(x, y, z)$ is constructed as

$$V(x, y, z) = \begin{cases} 1, & \text{if } h(x, y) \geq \frac{z}{Z_{max}} \\ 0, & \text{otherwise} \end{cases}$$

A polygonal mesh is extracted by applying marching cubes at an iso-level $\tau = 0.5$. Before exporting the finished model in the GLB format, which enables direct visualization via a web browser and a virtual reality headset.

5 Findings and Result

From an experimental point of view, as confirmed by the VRGen tool, the proposed method using the conditional GAN was able to learn the relationship between twodimensional architectural floor plans and their corresponding heightmap representations successfully. Following over 200 epochs of training, the loss function of the generator reached a stable point of between 21.7 and 24.6 and the loss function of the discriminator reached a value of just under 0.0000. The outputs that were generated stored the spatial data of the architectural plans. The production of various loss functions had a direct effect on the quality of output. The loss of the edge preservation, which rose up to almost 0.87 to 1.57, ensured a clear boundary among architectural buildings. The value of perceptual loss, which varied between 0.44 and 1.12, helped in improving the structural consistency by promoting similarity in the higher-level feature space. To guarantee accurate heightmap reconstruction the L1 reconstruction loss stabilized ranging from 0.63 to 1.55. Post-processing techniques such as denoising, wall refinement floor leveling gap closing and morphological filtering were then added to the heightmaps to give them an additional level of polish and remove noise on the surfaces as well as blur the boundaries. With the help of these improvements precise volumetric extrusion was made possible and all test samples had successful 3D mesh reconstruction with few topological errors. The finished models could be easily visualized in web-based and virtual reality environments because they were properly scaled and exported in .glb format. In general, the experiment results confirm the feasibility and effectiveness of the VRGen systems in automatic 2D-to-3D conversion when used in the virtual reality applications .

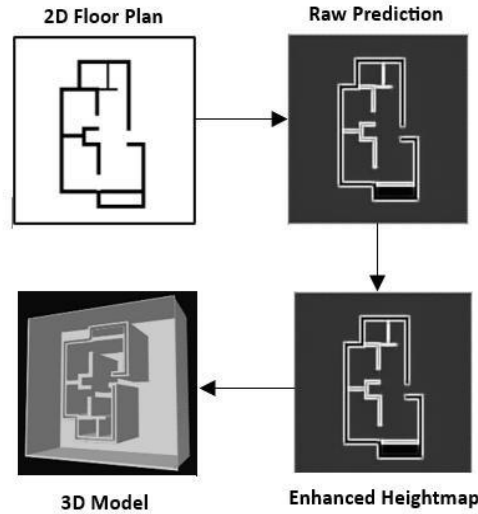


Fig. 5.1. VRGen 2D to 3D generation pipeline

6 Discussion

The obtained results suggest that pipelines with the use of deep learning can be effectively used in automating the two-dimensional architectural floor plans. To three-dimensional models to be presented in the visualization and virtual reality (VR) mode applications. The learning of an image was possible due to the use of a Pix2Pix conditional GAN directly based on paired floor plans and heightmap spatial and structural representation data, which confirms that methods of image-to-image translation are efficient [3], [4]. Among the principal opportunities of the proposed system is the elimination of manual 3D modeling activities. The conventional CAD and Solutions that are based on blender need the involvement of highly skilled individuals and time investment. The proposed pipeline, under training, will, on the other hand, auto-generate 3D models. This cut down in labor is in line with the recent research that encourages automating in real estate marketing [1],[5]. Direct export of models in the GLB format further adds deployment efficiency by allowing browser-based and VR visualization without the need for other software dependencies [6]. The voxel-based strategy used for mesh construction effectively balanced computational efficiency with geometric fidelity. While small surface irregularities emerged for complex floor plan configurations, the generated models were structurally coherent and supported smooth navigation. Similar limitations have been documented in earlier approaches for floor plan-driven reconstruction, related mainly to limited dataset size and resolution [2] and [7]. Increasing the variability of training data, together with multi-scale learning mechanisms, comprises a promising avenue toward enhancing the robustness of reconstruction as well as improving the quality of surfaces. The results suggest the suitability of the proposed framework in the context of initial architectural visualization, client-centered presentation, and walkthrough simulation.

7 Conclusion and Future work Discussion

In this research, the VRGen system uses a deep learning-based method for automating the process of converting two-dimensional architectural floor plans into three-dimensional models intended for use in virtual reality walkthroughs. This is achieved via a conditional Generative Adversarial Network architecture. The system generated the heightmap outputs that were structurally consistent and spatially well-aligned by combining multiple loss components with a dedicated post-processing pipeline. Such sophisticated heightmaps enabled the reconstruction of three-dimensional meshes and the generation of GLB models with the retention of the architectural structure. Experimental results showed the efficacy of the interactive generation of three-dimensional environments from two-dimensional floor plans, with proof-of-concept performance despite the constrained data. VRGen offers significant advantages for the visualization of properties, initial architectural designs, and teaching purposes through immersion within the environment prior to implementation. Moreover, the automated process enhances the efficient use of time and resources in support of sustainable development. The next generation of work will involve increasing the volume of data to encompass a wider range of architectural styles, incorporating semantic coding of the interior components, and improving the realism with the help of the texturegeneration and material-modelling tools. Also, it may also be enhanced by introducing real time user interaction and multi-modal input like text description or drawings to enhance the flexibility of a system and the scope of application. Moreover, the future studies can be specialized on the model design optimization to be brought closer to inference both in reduced time and lower cost, making it ready to be deployed to resource-constrained devices. The practical relevance and strength of the proposed system can also be enhanced in another way, with the large-scale application and use of the proposed system in reality.

References

1. R. G. Kippers, M. Koeva, M. van Keulen, S. J. Oude Elberink: Automatic 3D building model generation using deep learning methods based on CityJSON and 2D floor plans. ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences (2021).
2. Z. Zhang, X. Wu, Y. Liu: 3D residential reconstruction from a single indoor floor plan. IEEE Trans. Vis. Comput. Graphics (2021).
3. S. Park, H. Kim: 3DPlanNet: Generating 3D models from 2D floor plan images using ensemble methods. Electronics, vol. 10, no. 22, p. 2729 (2021).
4. A. C. Barreiro, M. Trzeciakiewicz, A. Hilsmann, P. Eisert: Automatic reconstruction of semantic 3D models from 2D floor plans. Fraunhofer HHI / Humboldt University of Berlin, Berlin, Germany. <https://arxiv.org/abs/2306.01642>
5. V. K. Reddy, G. N. V. Prajwal, V. G. Reddy, N. Srushti, N. S. Dr. Nirmala S.: Plan-MORPH: Conversion of 2D Floorplans to 3D Models. ER Publications (2020). https://www.erpublishings.com/uploaded_files/download/venkat-krishna-reddy-g-n-vprajwal-venu-gopal-reddy-n-srushti-g-s-dr-nirmala-s_KSRLX.pdf
6. P. Isola, J.-Y. Zhu, T. Zhou, A. A. Efros: Image-to-image translation with conditional adversarial networks. In: Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR) (2017).
7. A. Kazlouski : Floor plans generation via GANs. Czech Technical University (2022). <https://www.fa.cvut.cz/cs/galerie/atelierove-prace/34713-floor-plan-generation-gans>
8. A. Kesar: Text-to-floorplan using GANs. Kaggle Notebook (2020). <https://www.kaggle.com/code/aarykeskar/text-to-floorplan-using-gans>
9. Generative AI and HCI (2025). <https://dl.acm.org/doi/epdf/10.1145/3706599.3707213>
10. Y.-K. Juan, H.-H. Chen, H.-Y. Chi: Developing and evaluating a virtual reality-based navigation system for pre-sale housing sales. Applied Sciences, vol. 8, no. 6, p. 952 (2018).
11. The implementation of virtual reality (VR) technology in the real estate industry. ResearchGate (2023). https://www.researchgate.net/publication/372676021_THE_IMPLEMENTATION_OF_VIRTUAL_REALITY_VR_TECHNOLOGY_IN_REAL_ESTATE_INDUSTRY
12. Interactive visualization and virtual reality for construction and real estate. In: Proc. CONVR (2013). <https://itc.scix.net/pdfs/convr-2013-52.pdf>
13. 3D reconstruction and visualization techniques for built environments. ISPRS Archives, vol. XLVI-4/W4-2021, pp. 49–55 (2021).
14. Generating 3D architected nature-inspired materials and granular media using diffusion models based on language cues. ResearchGate (2022). https://www.researchgate.net/publication/365758918_Generating_3D_architected_nature-inspired_materials_and_granular_media_using_diffusion_models_based_on_language_cues
15. O. Ronneberger, P. Fischer, T. Brox: U-Net: Convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI) (2015).
16. K. Li et al.: Learning to predict 3D meshes from a single image via depth consistency. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR) (2018).
17. I. Sobel, G. Feldman: A 3×3 isotropic gradient operator for image processing. Stanford AI Project (1968).
18. L. Rudin, S. Osher, E. Fatemi: Nonlinear total variation based noise removal algorithms. Physica D, vol. 60, pp. 259–268 (1992).
19. J. Johnson, A. Alahi, L. Fei-Fei: Perceptual losses for real-time style transfer and superresolution. In: Proc. Eur. Conf. Comput. Vis. (ECCV)

Acknowledgements

We would like to express our sincere gratitude to **Prof. Vijesh Mundokalam** for his invaluable guidance and mentorship throughout this project. His consistent support and technical insights were instrumental in troubleshooting challenges and ensuring the work remained on schedule. We also extend our thanks to **Prof. Sayali Badhan**, Project Co-ordinator, and **Dr. Jaya Gupta**, Head of the Department, for their cooperation and essential administrative support. Furthermore, we appreciate the assistance provided by the faculty and staff of the AIML Department during the various stages of this work.

Finally, we are grateful to our Principal, **Dr. Uttam Kolekar**, and the college administration for providing the necessary facilities and resources. We would also like to acknowledge our classmates and everyone else who contributed directly or indirectly to the successful completion of this project.

(Yash Penkar 22106118)

(Prarthana Patil 22106035)

(Rutuja Pawar 22106043)

(Dhruv Sawant 22106015)

Date: